

Distributed Systems

DAT520 - Spring 2026

Chapter 2 - Basic Abstractions

Prof. Hein Meling

Basic Abstractions

- Processes — abstracting computational entities
 - Physical *computer*, *virtual machine*, or even a specific *thread of execution*
- Links — abstracting the network
- Failure detectors — abstracting time
 - Capture reasonable timing behaviors of proc.es and links

Processes

- Assume a finite set $\Pi = \{p_1, \dots, p_N\}$
- $N = |\Pi|$ is the number of processes (system size or cardinality)
- Each process **P** models a *sequential program* (no threads)
- Processes (**P**s) may maintain application or protocol *state*

What is the state of a Go
type, or Python/C#/Java
class?

State: The mutable fields
of the class/type

Processes and Messages

- **P**s have unique identities and know each other
- **P**s communicate by exchanging messages
 - Messages uniquely identified: $\langle \text{SenderID}, \text{SequenceNumber} \rangle$
- All **P**s in the system run the same *local algorithm*
 - The sum of these local algorithms constitutes the **distributed algorithm**.

Automata and Steps

- The execution of a distributed algorithm is represented by a sequence of steps executed by the **P**s.
- A **P** step consists of:
 - Receiving a message from another **P** (global event)
 - Executing a local computation (local event)
 - Sending a message to some **P** (global event)

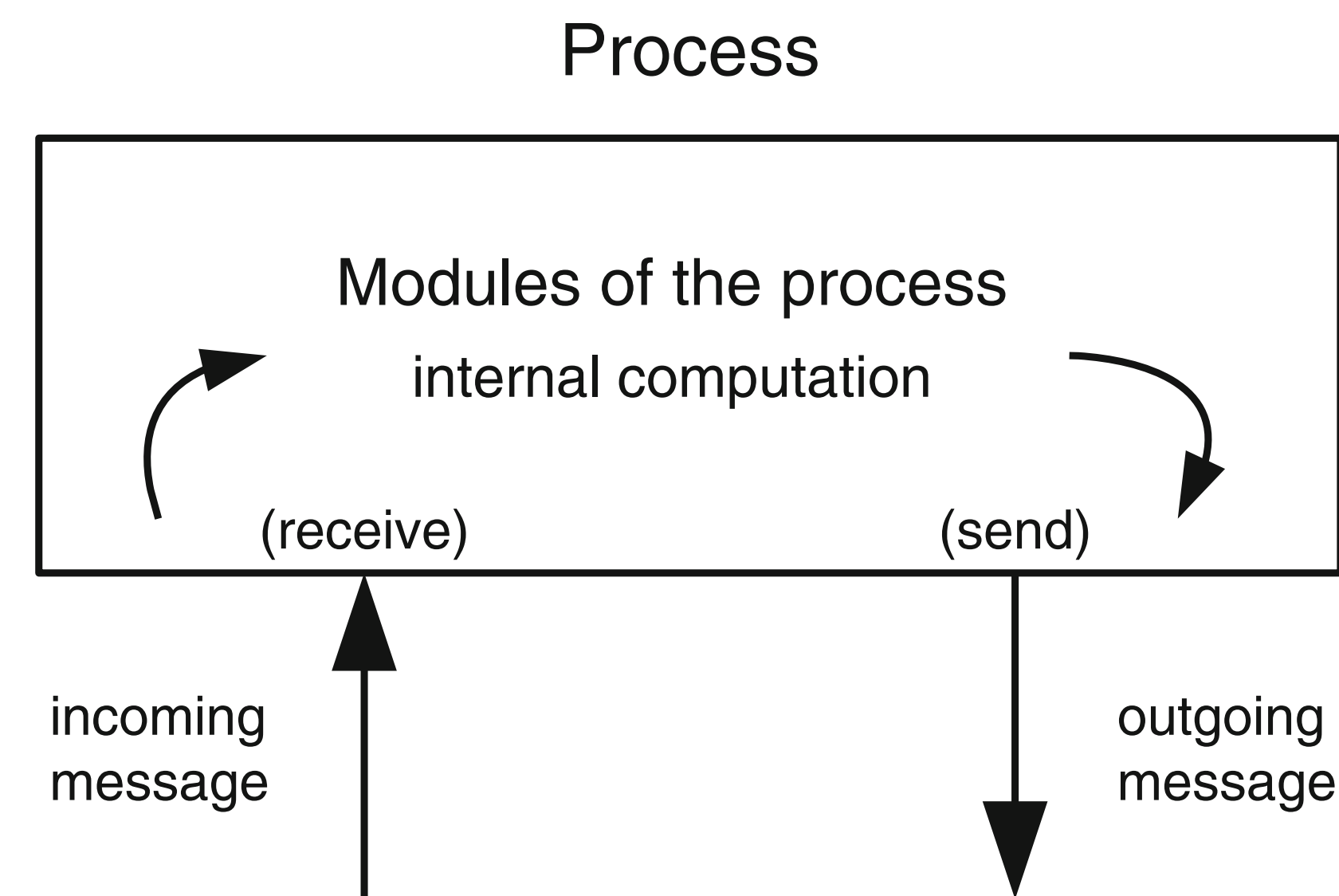


Figure 2.2: Step of a process

Automata and Steps

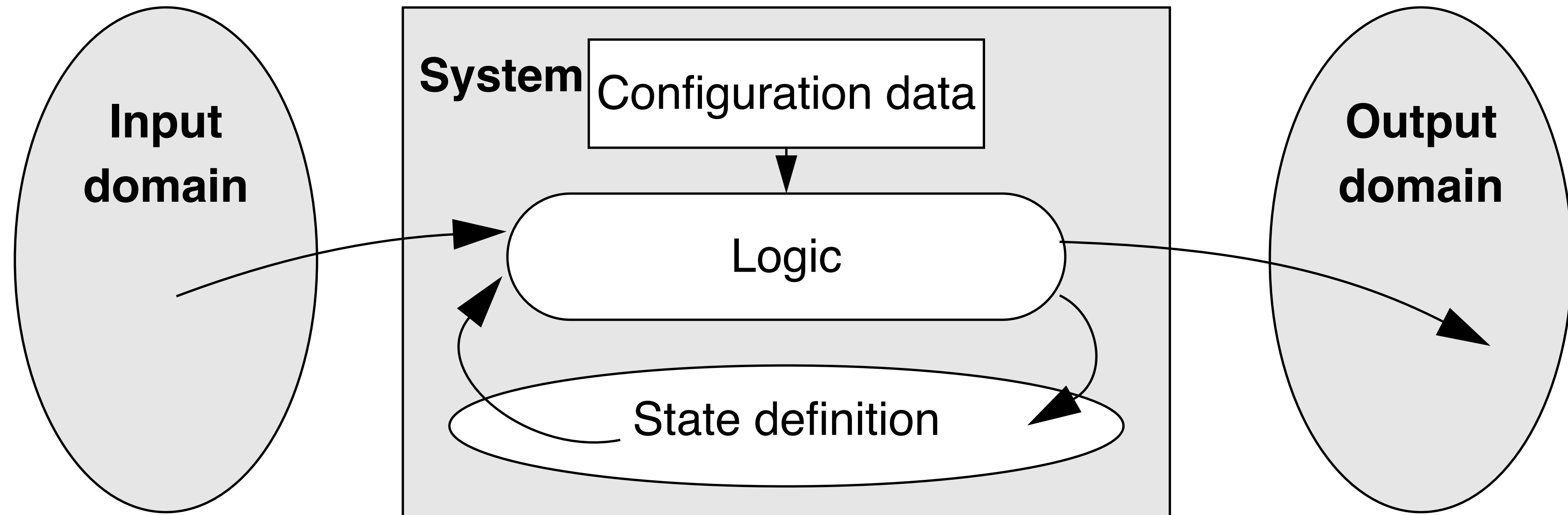


Figure 1.7 Simple model of a system

Automata and Steps

- Def. **Communication Step**:
 - A **P** sends a message to another, and the latter receives this message
- # of communication steps:
 - Reflects the latency of an algorithm

Safety and Liveness

- A **safety** property requires that the algorithm *cannot do anything wrong*
 - The algorithm is correct (correctness)
- Example: **Reliable links safety property**
 - No **P** should receive a message unless it was sent
 - The communication link should not invent messages out of thin air

Not enough to only
provide safety properties

Preventing bad things
from happening is easy

Simply do nothing!

Safety and Liveness

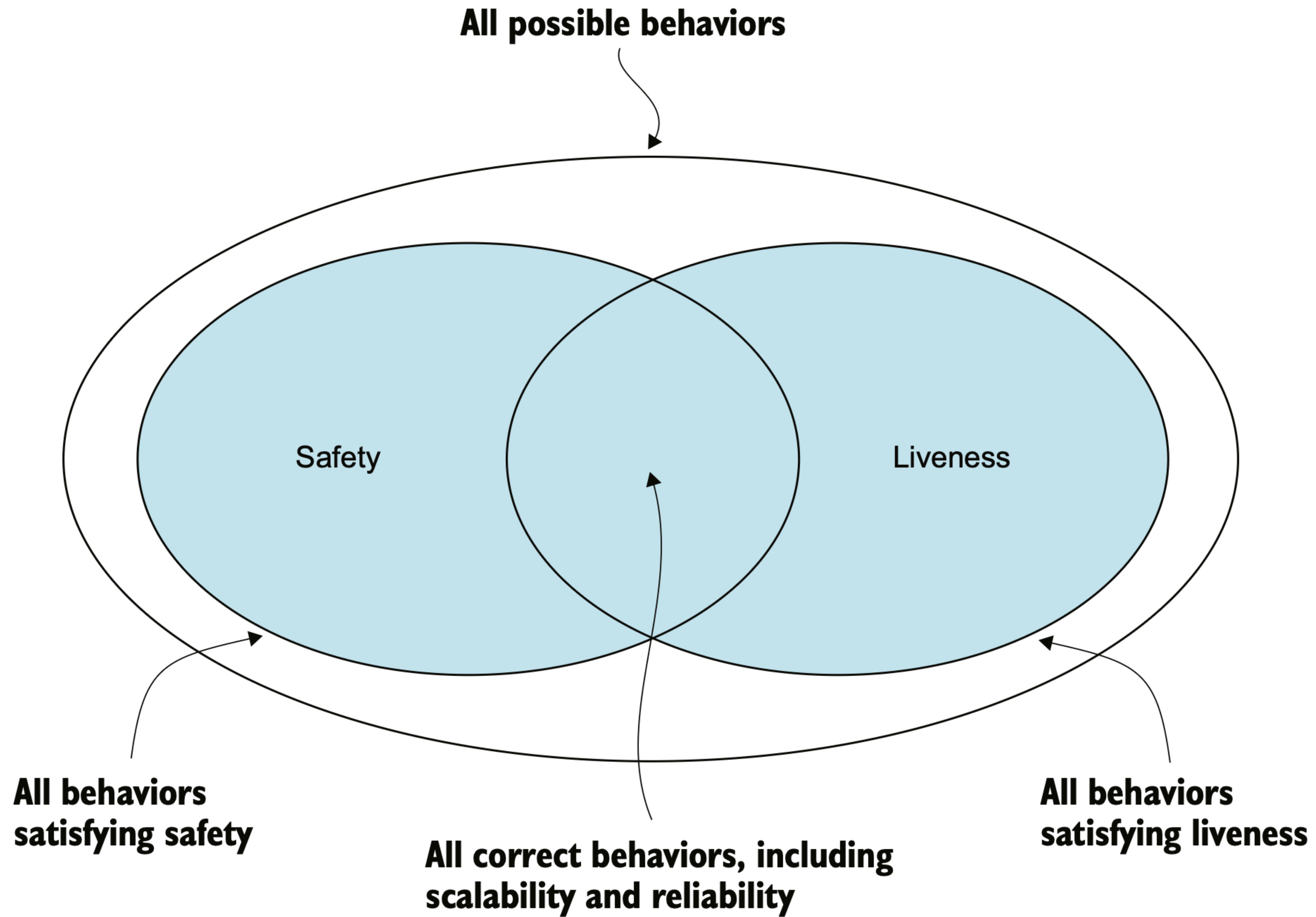
- A **liveness** property requires that the algorithm *do useful stuff*
 - Eventually, something good happens. Eventually = sooner or later
 - The algorithm makes progress
- Example: **Reliable links liveness property**
 - If a correct **P** sends a message to a correct destination **P**, then the destination **P** should eventually deliver the message
 - The communication link should not drop the message infinitely often

We want both
safety and liveness

Impossible to
guarantee both

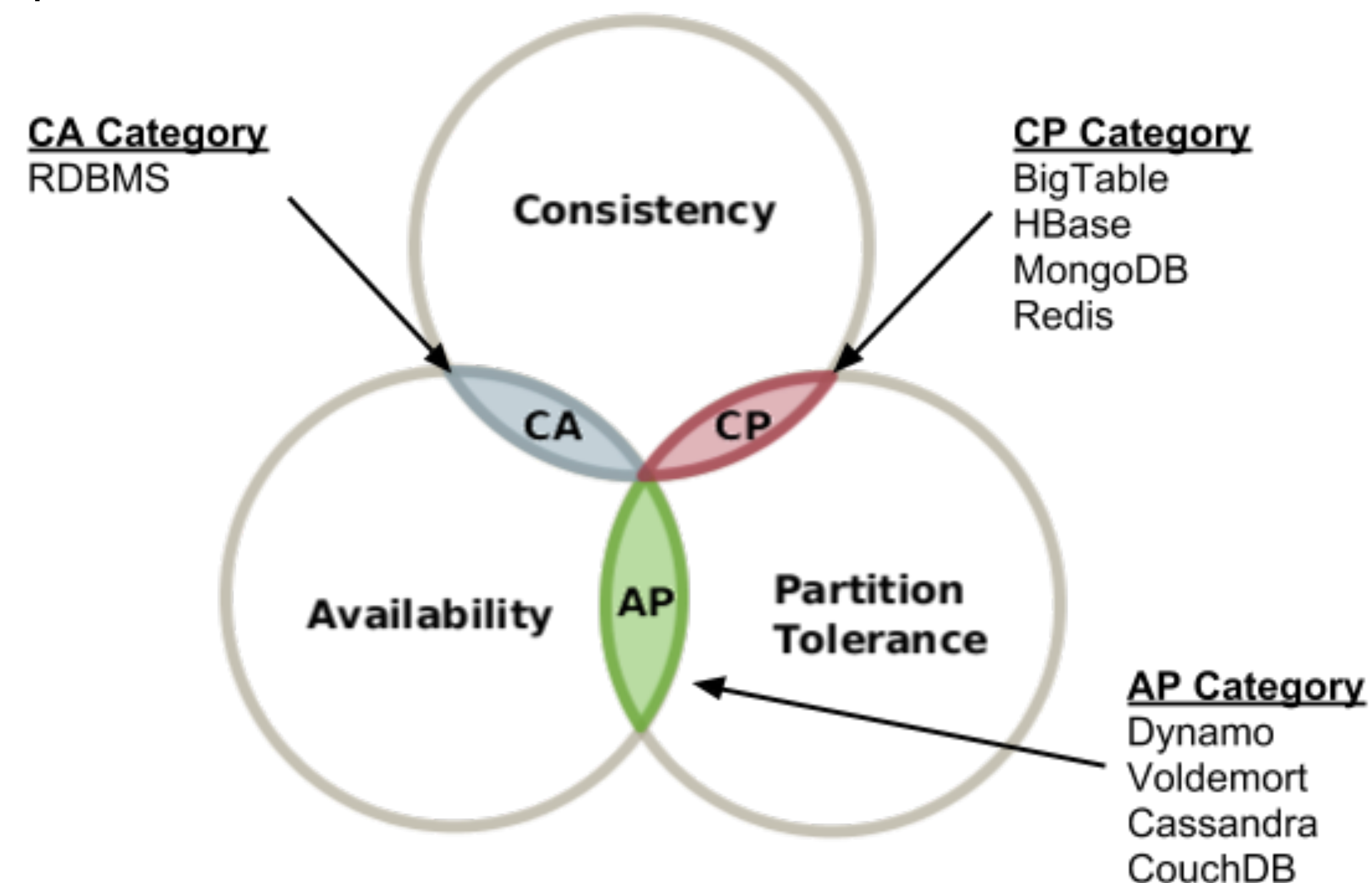
Tradeoff: Always safe,
but not always live

Tradeoff: Eventually safe,
and (almost) always live



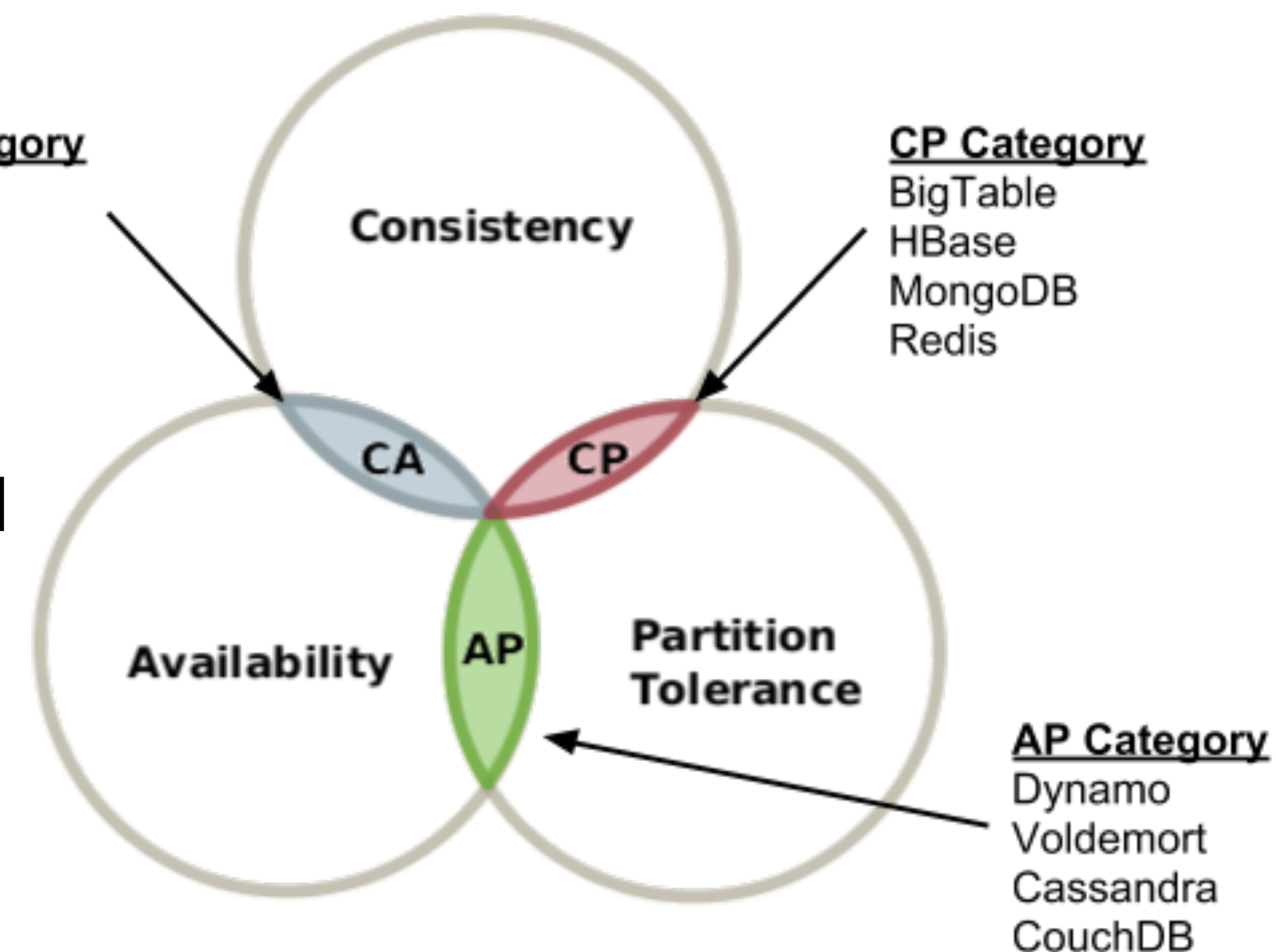
CAP Theorem (Wikipedia, 2019)

- *Impossible* for a distributed data store to provide more than **two** of the **three** guarantees:
 - **C**onsistency
 - **A**vailability
 - **P**artition tolerance



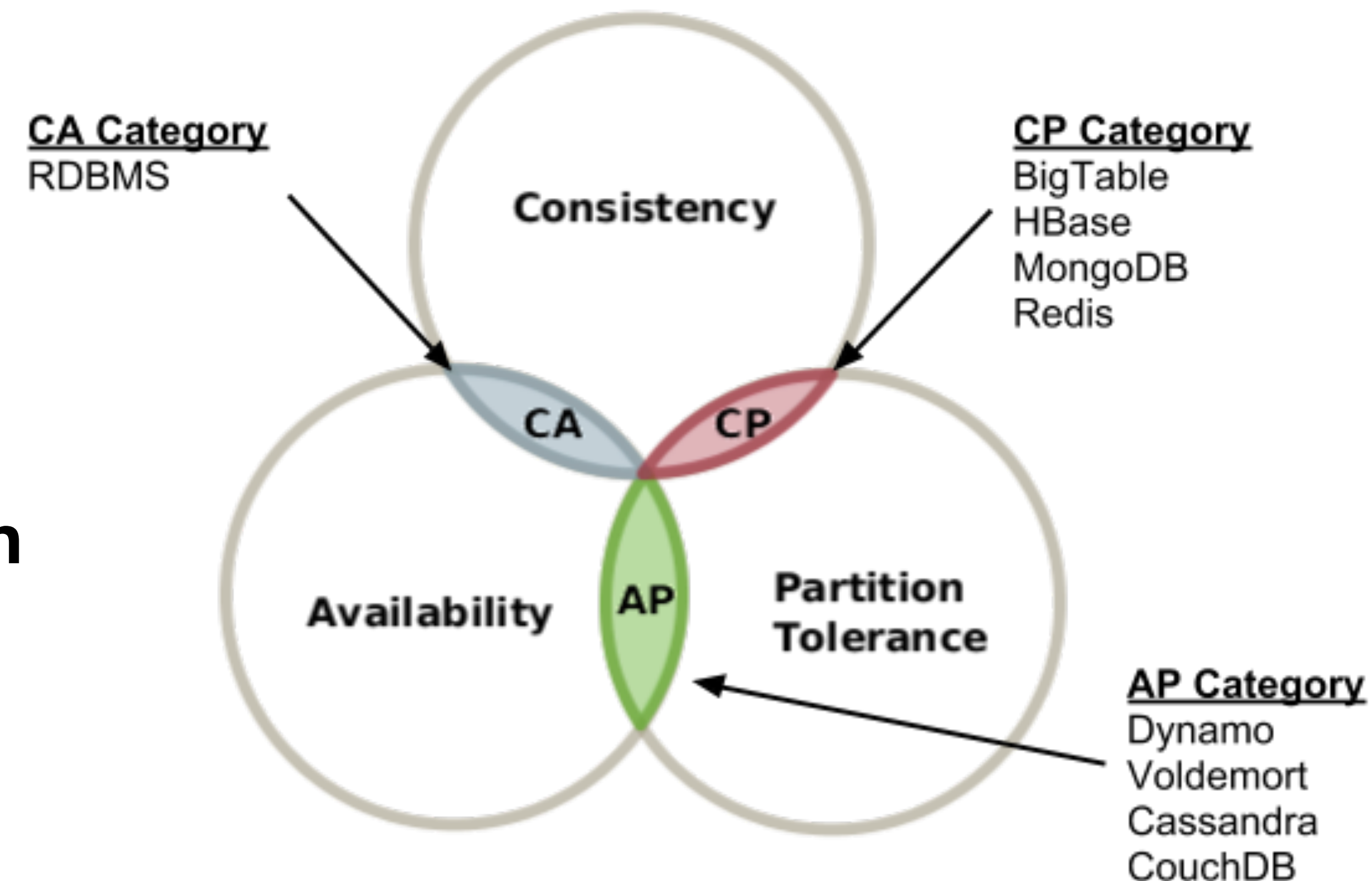
CAP Theorem

- In the presence of a **network partition**, a distributed system must choose between:
 - **C**onsistency
 - **A**vailability
- **P**artition tolerance is unavoidable in distributed systems



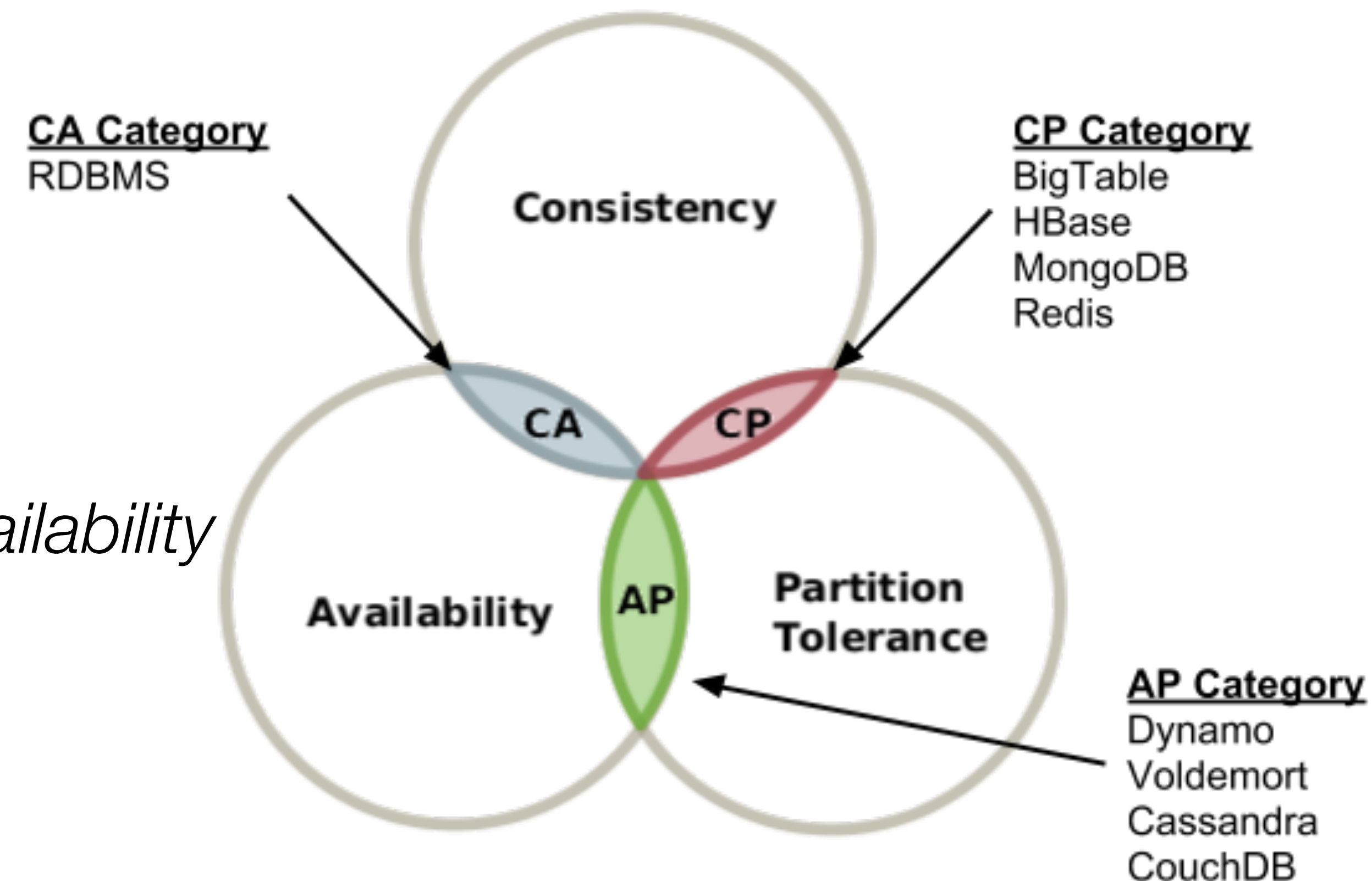
CAP Theorem: Network Partitions Are Inevitable

- No distributed system is immune to network failures
 - **P**artition tolerance *must always be assumed*
- The real trade-off is
 - **What to sacrifice during a partition**



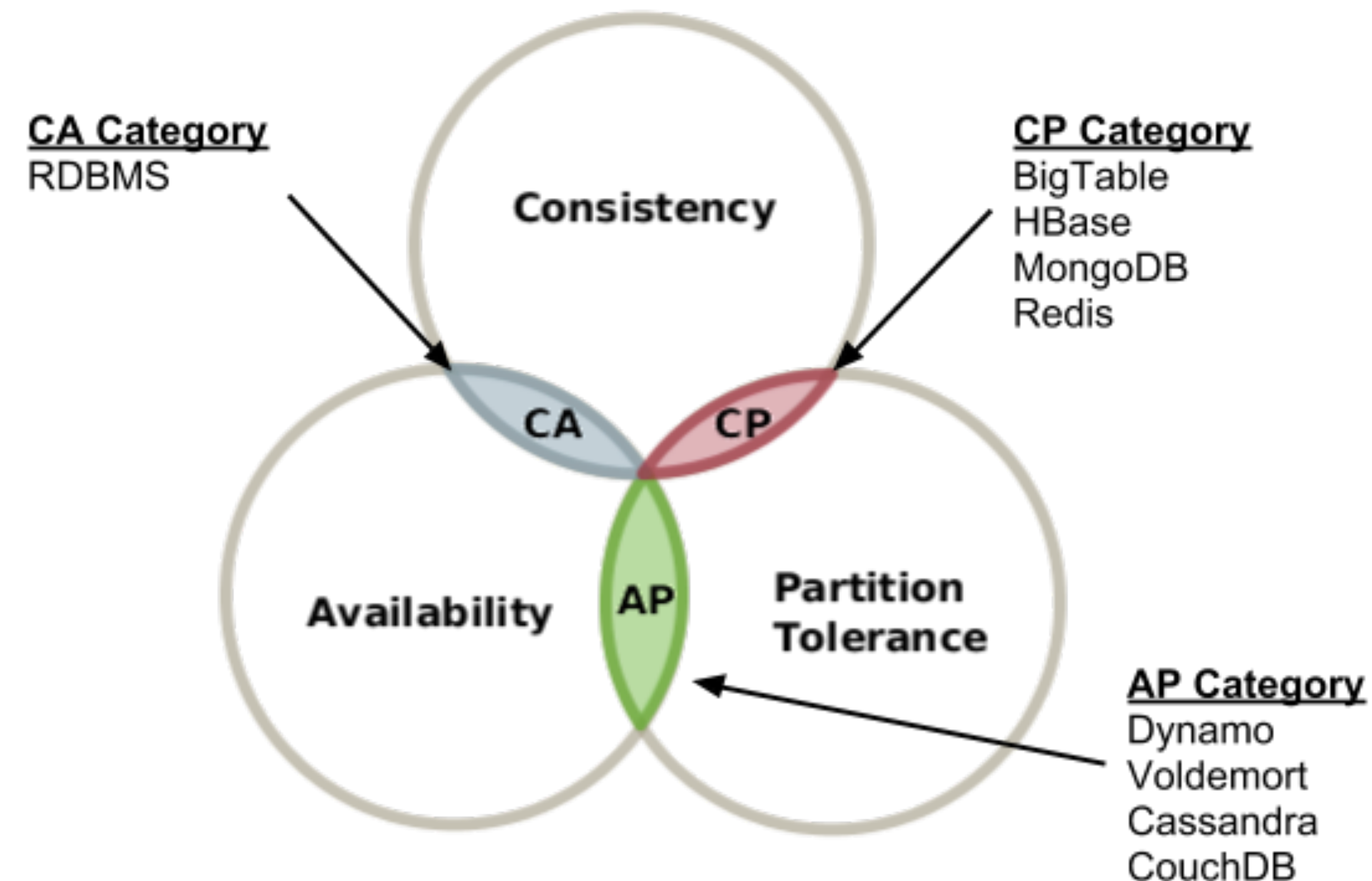
CAP Theorem: Consistency over Availability (CP)

- Prioritize returning:
 - *Correct and up-to-date* data
- **If** *correctness cannot be guaranteed*:
 - Return an error or a timeout
- Clients may experience *temporary unavailability*



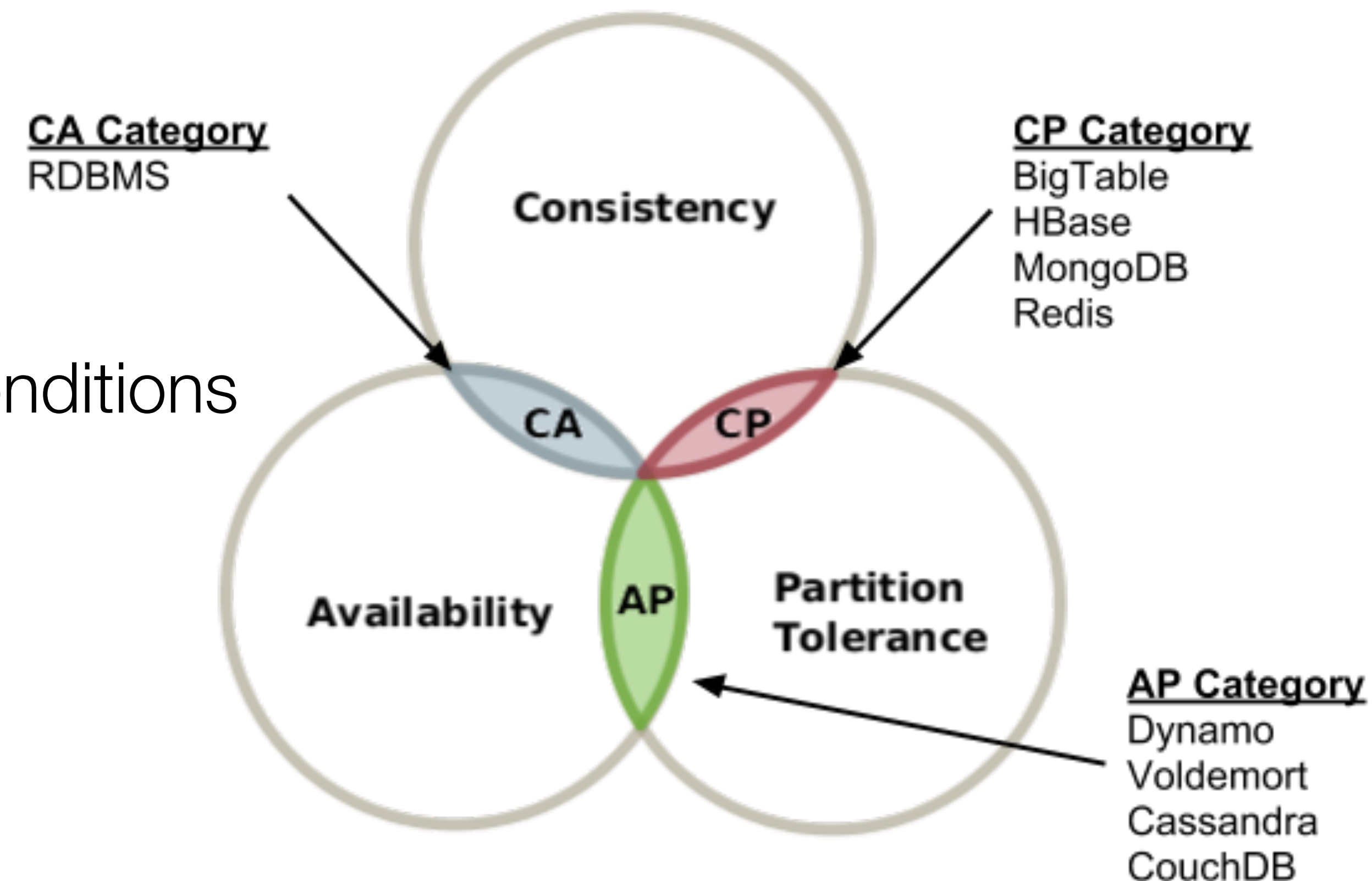
CAP Theorem: Availability over Consistency (AP)

- Prioritize returning:
 - Response to every request
 - The most recent available data
- Correctness is *not guaranteed* during a partition



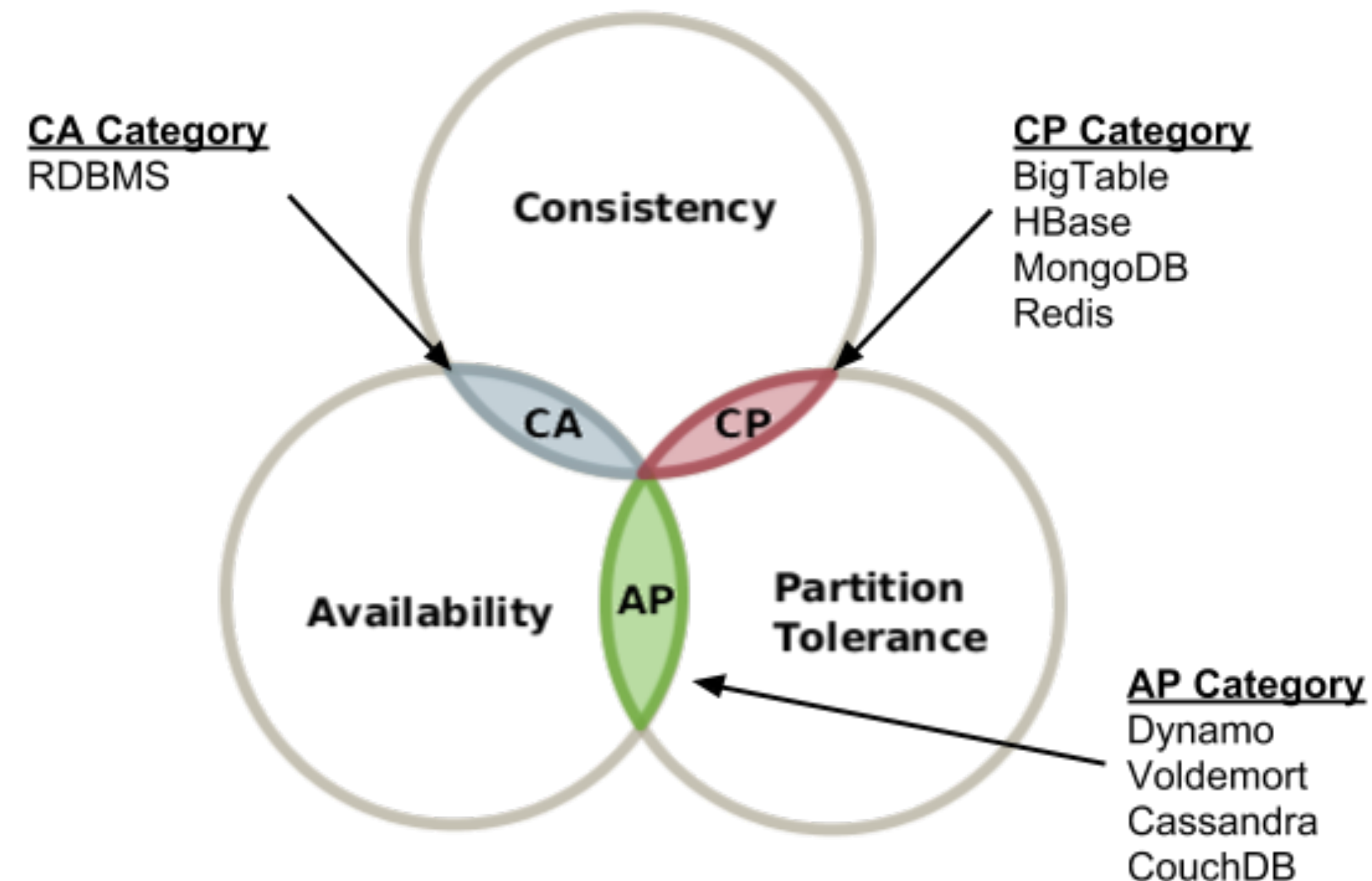
CAP Theorem: Normal Operation (No Partition)

- When the network is healthy:
 - **C**onsistency and
 - **A**vailability can be satisfied
- *No trade-off is required* under normal conditions



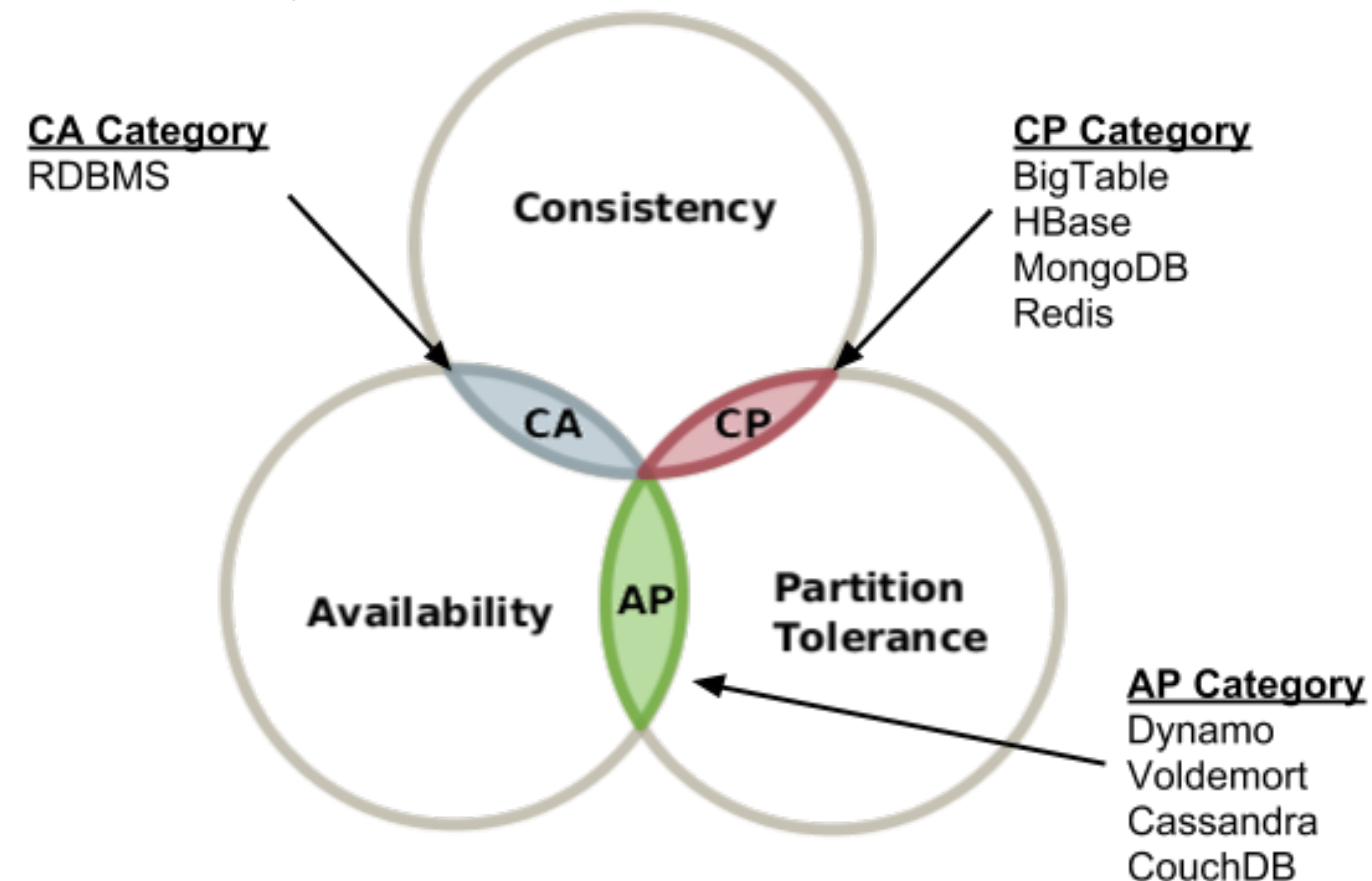
CAP Theorem: Common Misunderstanding

- **CAP** is often misinterpreted as:
 - “*You must permanently give up one property.*”
- This is **incorrect**



CAP Theorem: What CAP Actually Says

- The trade-off between **C**onsistency and **A**vailability
- Applies *only* during network partitions
- Outside of failures:
 - Systems can provide both



Joke time!

ADD JOKES/CARTOON

Abstracting Processes

Process Failures

Process Failures

- Definition:** A **failure** occurs whenever a process does not behave according to its algorithm

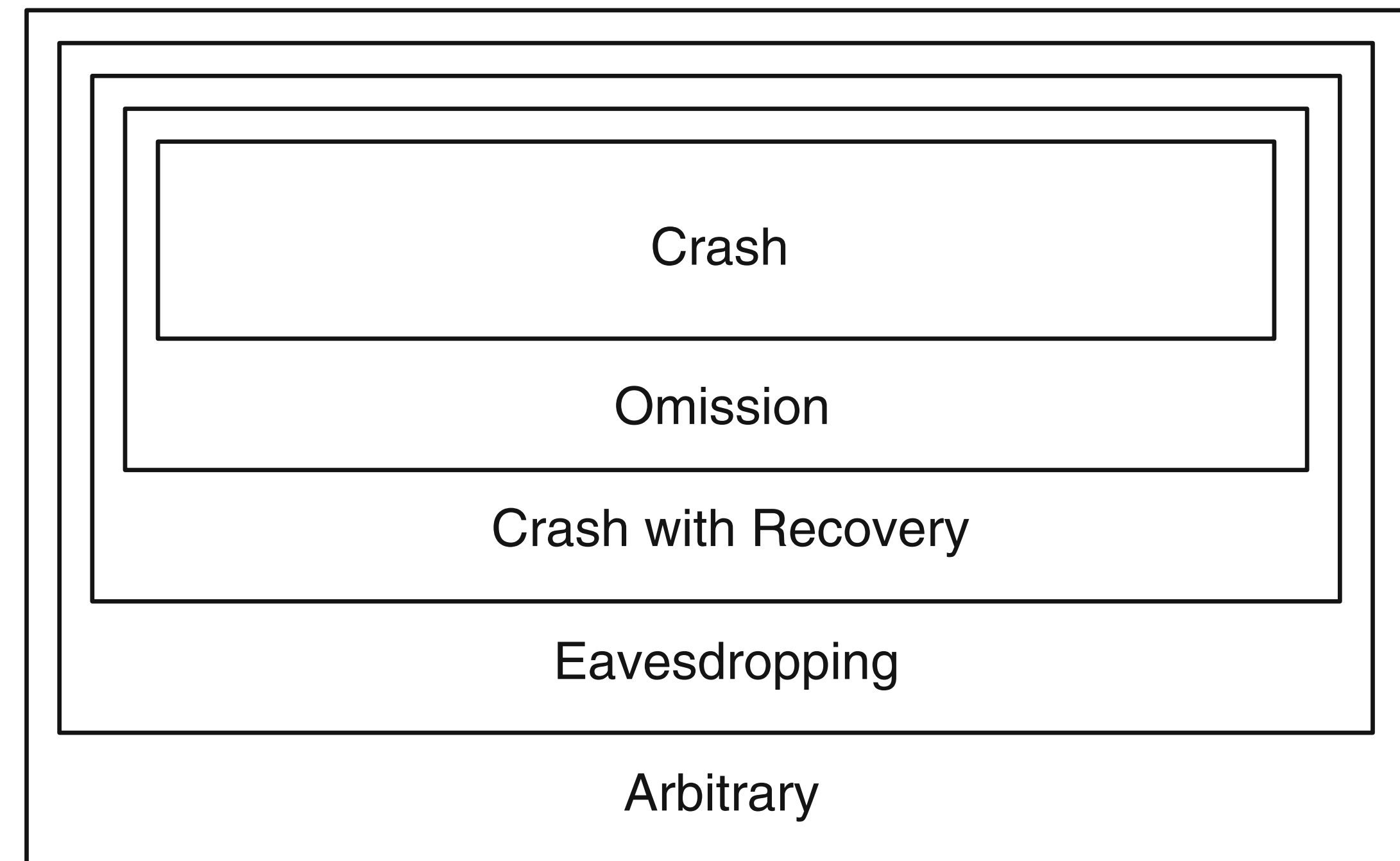


Figure 2.3: Types of process failures

Process Failures: Unit of Failure

- The unit of failure is a process (**P**)
- Not:
 - machine
 - object
 - thread
 - operating system

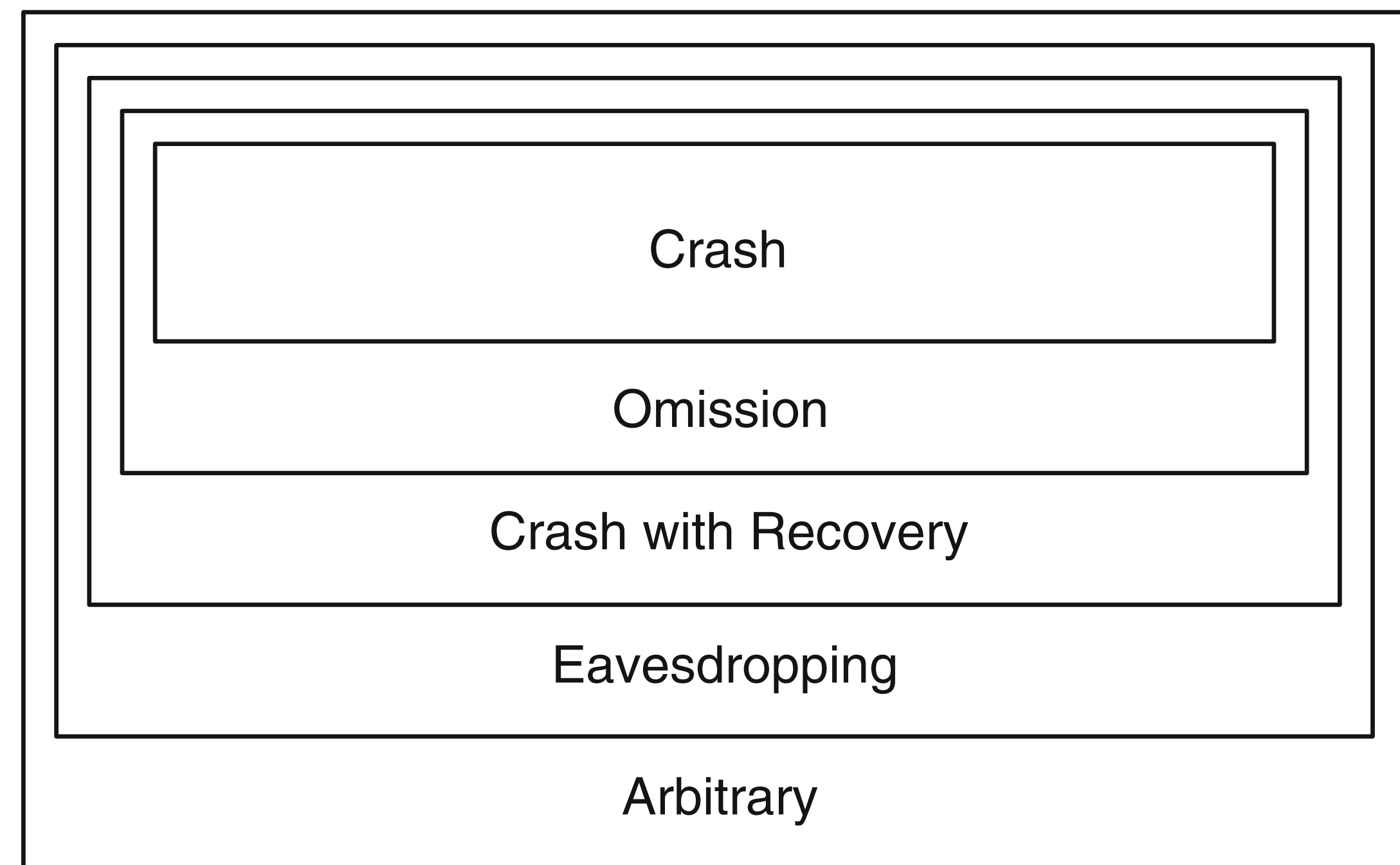


Figure 2.3: Types of process failures

Process Failures: Process-Level Failure Semantics

- When a **P fails**, *all its components fail simultaneously*
 - e.g., all threads within the **P**
- The failure is **atomic at the process level**

This is just an **assumption** we make!

Process Failures: In Practice

- In real systems, only a single thread may fail
 - e.g., a worker thread crashes due to an exception
 - Other threads may remain active

Process Failures: In Practice

- Example
 - Ping-response thread is still running
 - The worker thread has failed
- Consequence
 - Other **P**s may not detect the failure, since the **P** still responds to pings

Crash Failures

Crash Failure

- A **P** executes its algorithm correctly
 - Until some time t
 - After time t :
 - stops all local computation
 - sends no messages to any other **Ps**

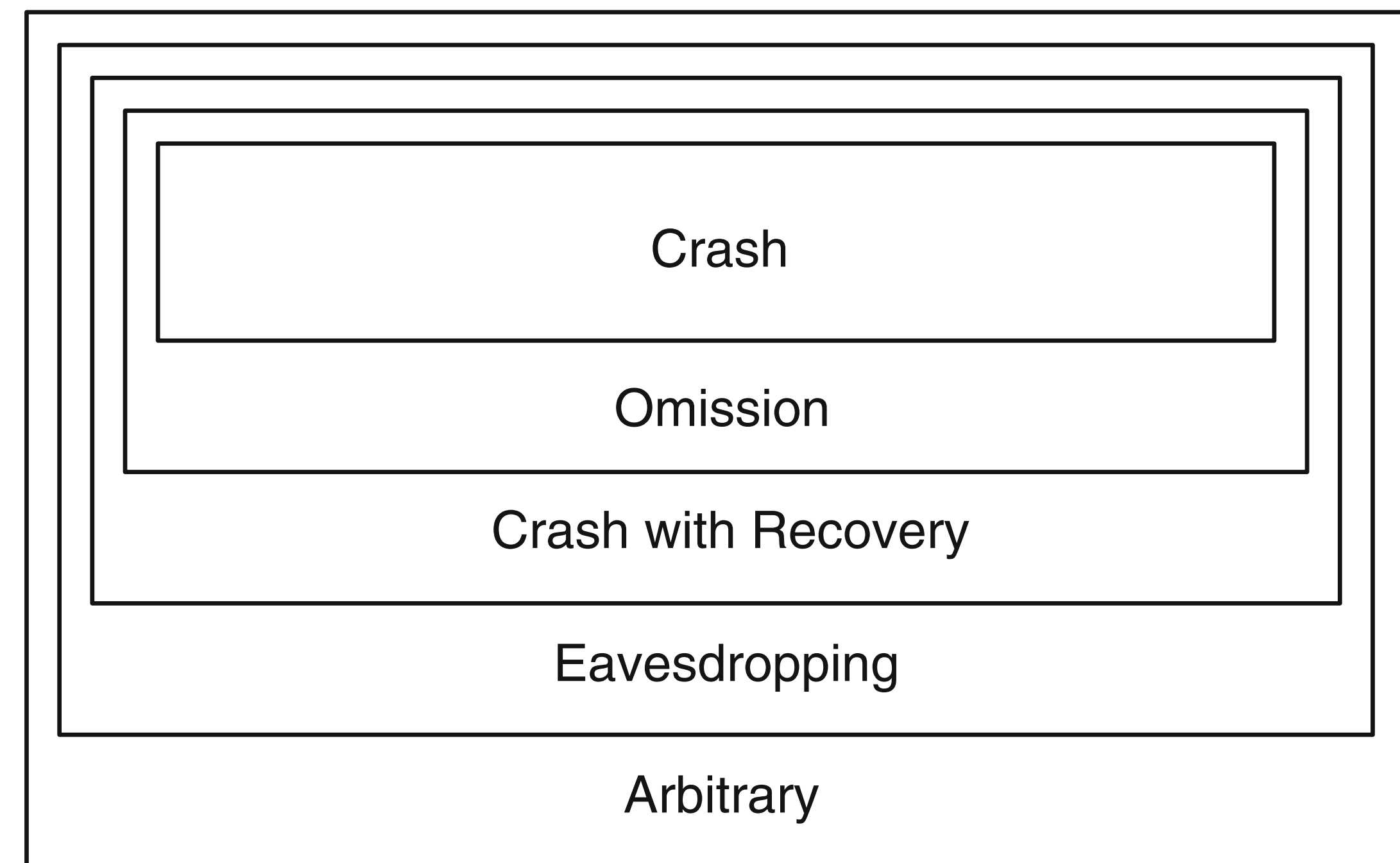


Figure 2.3: Types of process failures

Crash Failure

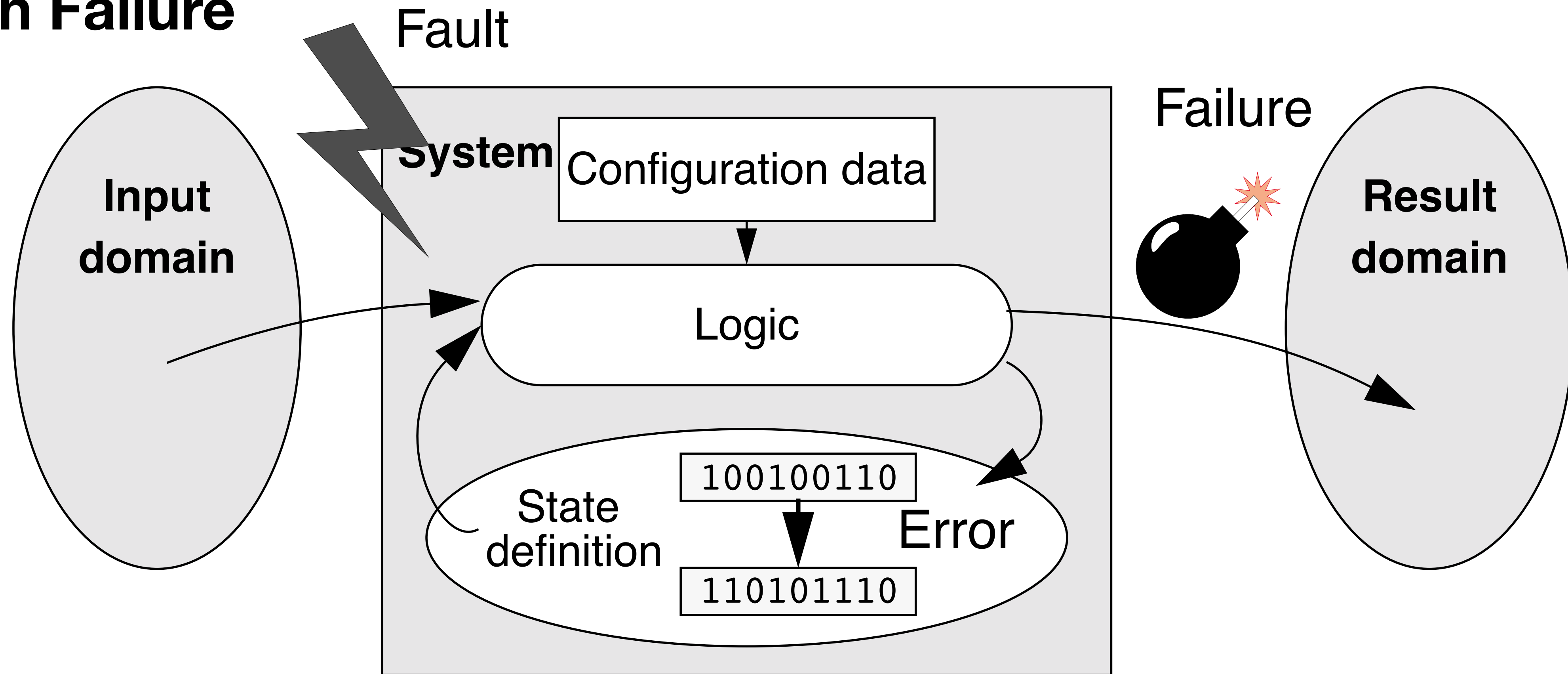


Figure 1.8 Illustration of the fault, errors and failure concept in the Moore/Mealy like system model

Crash Failure: Crash-Stop Process Abstraction

- Once a **P** crashes, it never recovers
- **P** remains permanently stopped after the crash

Crash Failure: Faulty vs. Correct Processes

- **Faulty** Process
 - crashes during execution
- **Correct** Process
 - never crashes
 - executes an infinite number of steps

Omission Failures

Omission Failure (Fault)

- **Definition:** *A process fails to send or receive a message that it is supposed to, according to its specification*
- Common causes:
 - Network congestion or network outages
 - Buffer overflows in routers
 - Buffer overflows in end systems

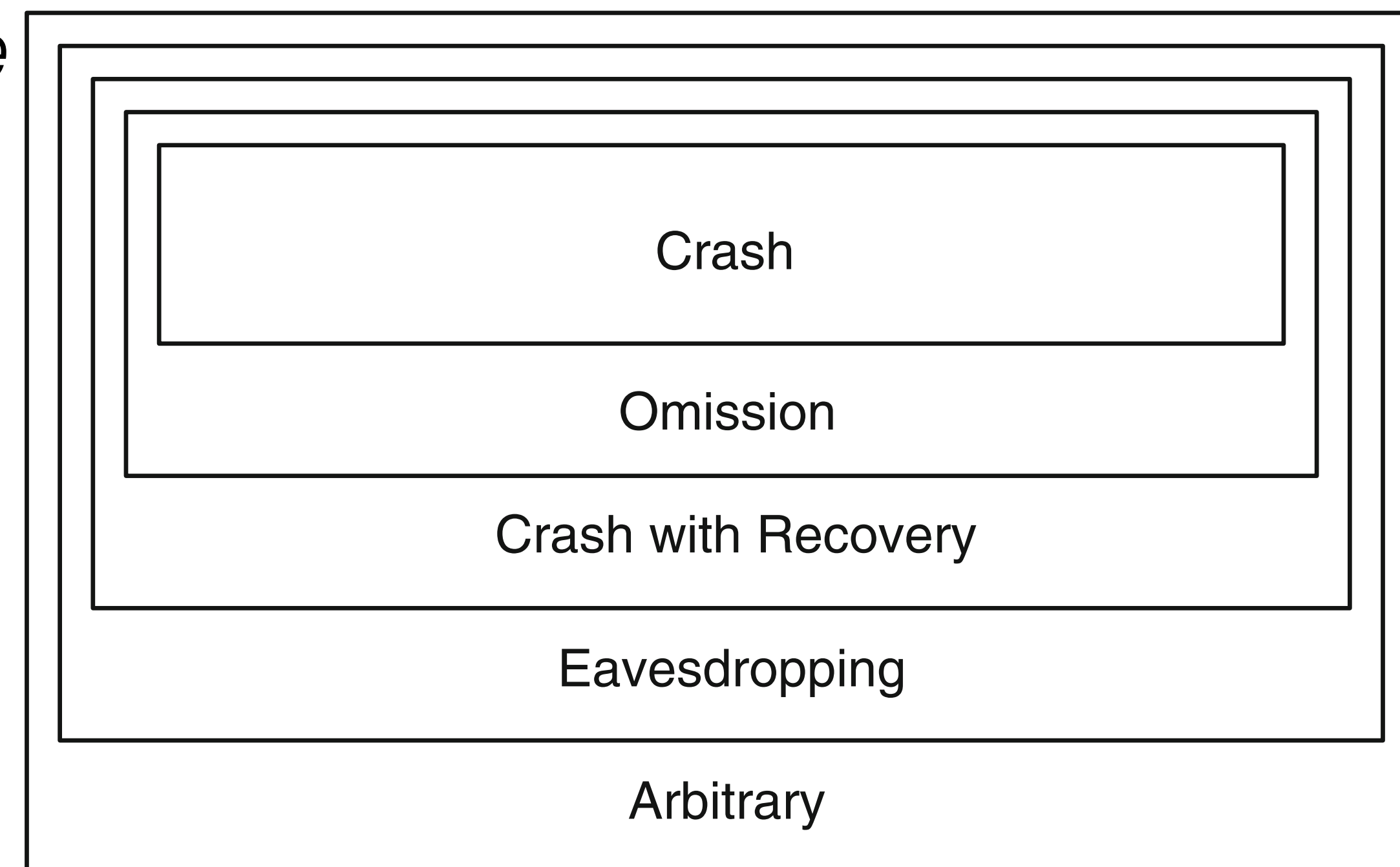


Figure 2.3: Types of process failures

Crashes and Recoveries

Crash with Recovery

- **Definition:** *A process may crash and later recover*
- Unlike crash-stop, *recovery is explicitly allowed*

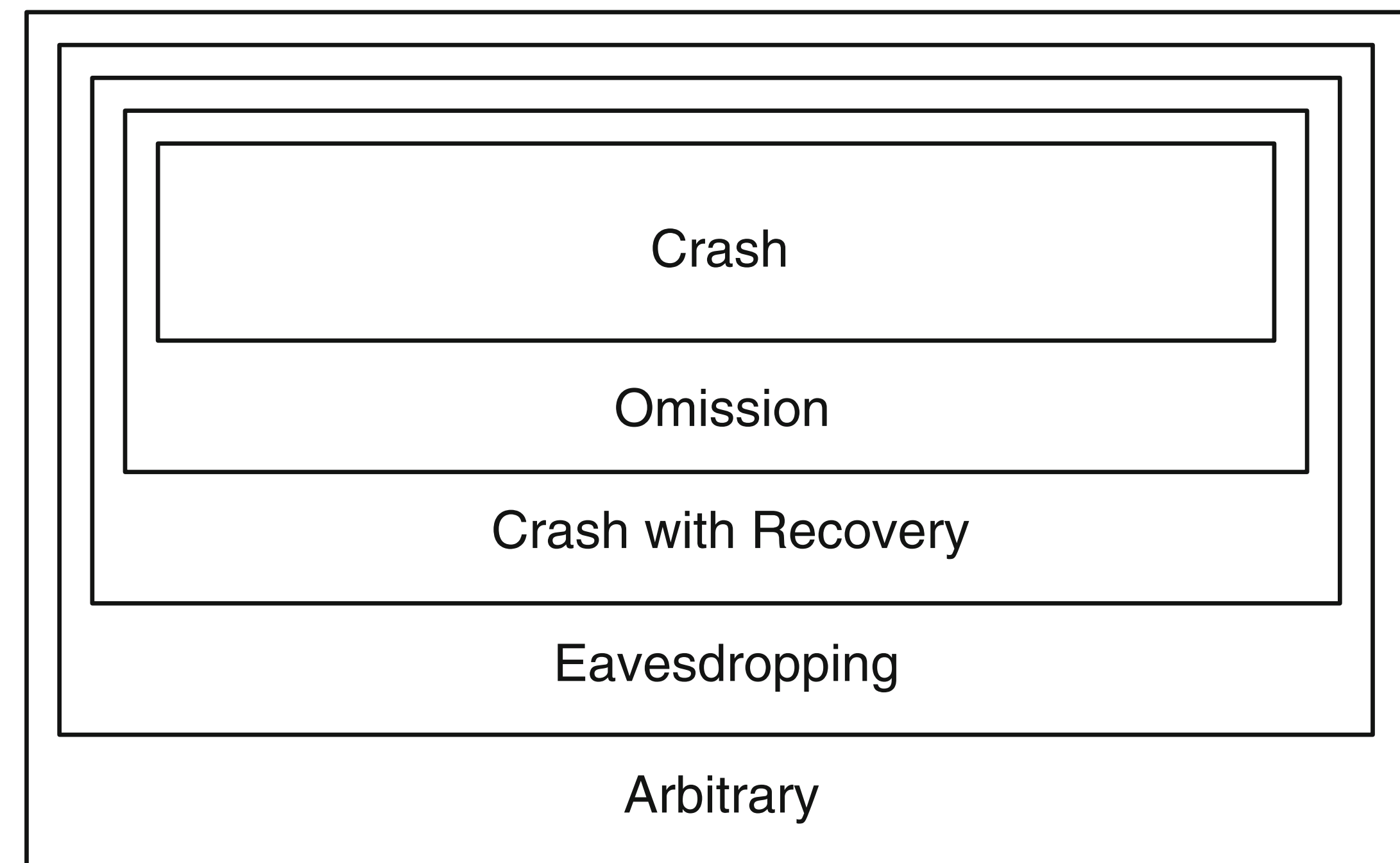


Figure 2.3: Types of process failures

Crash with Recovery: Crash–Recovery Process Abstraction

- Process is **faulty** if *it crashes* and
 - never recovers, or
 - recovers infinitely often
- Process is **correct** if it *eventually*
 - remains up and running forever

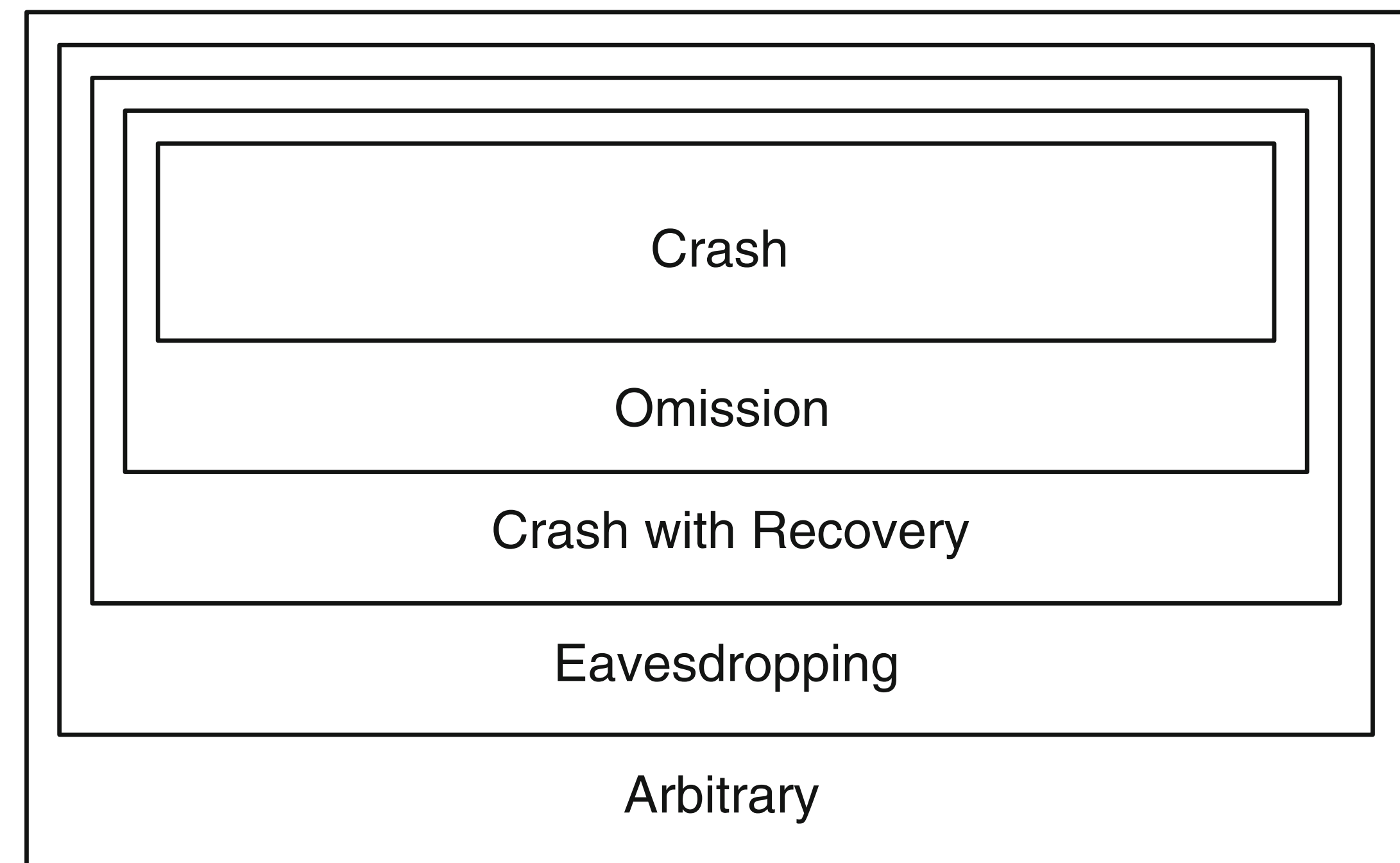


Figure 2.3: Types of process failures

Crash with Recovery: State Persistence

- Recovery requires
 - *preserving the state across crashes*
- Use stable storage (e.g., disk or SSD)
- Ensures **P** can resume correctly after recovery

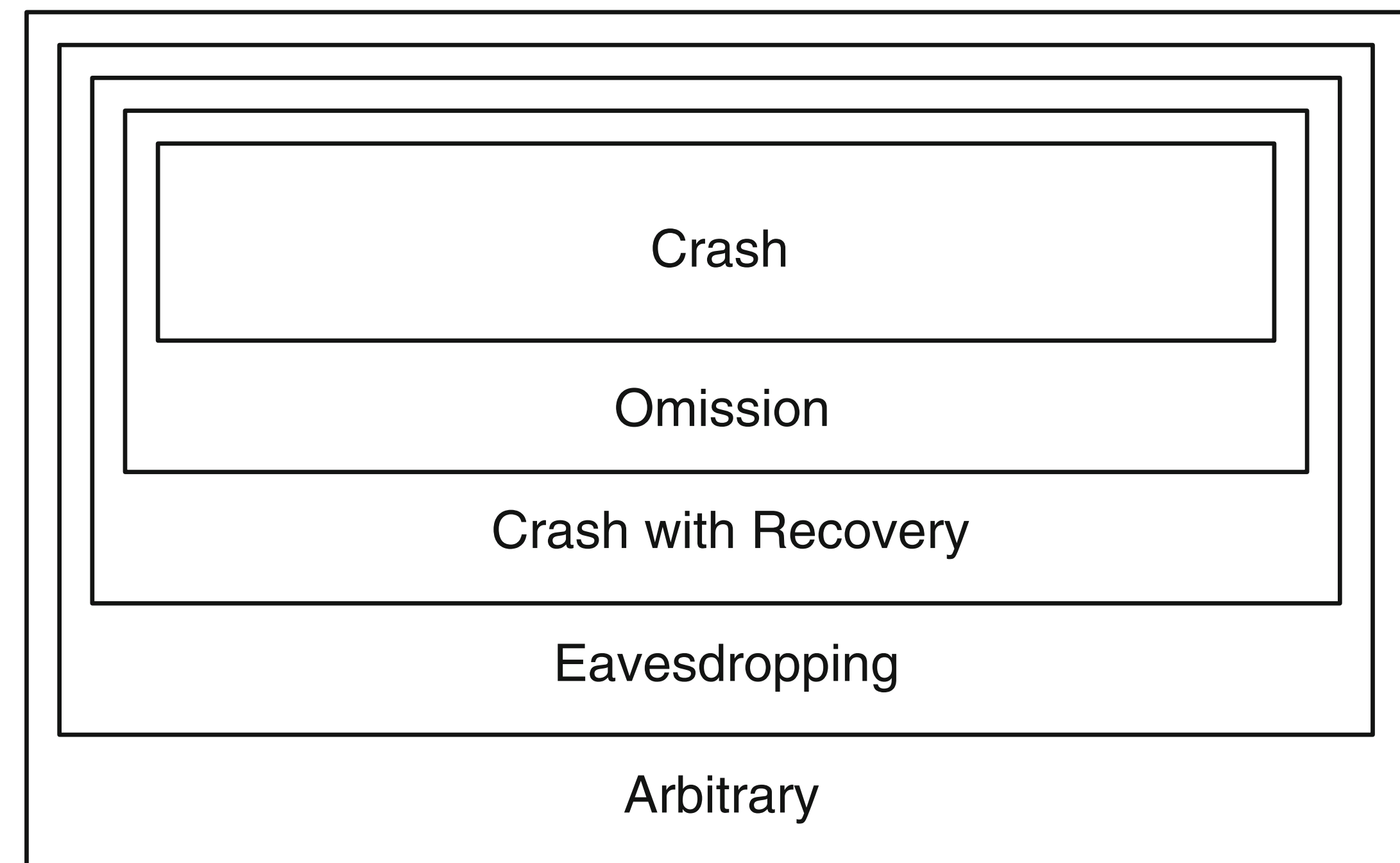


Figure 2.3: Types of process failures

Eavesdropping

Eavesdropping Faults

- Passive faults where communication is observed
- Typically handled using cryptography

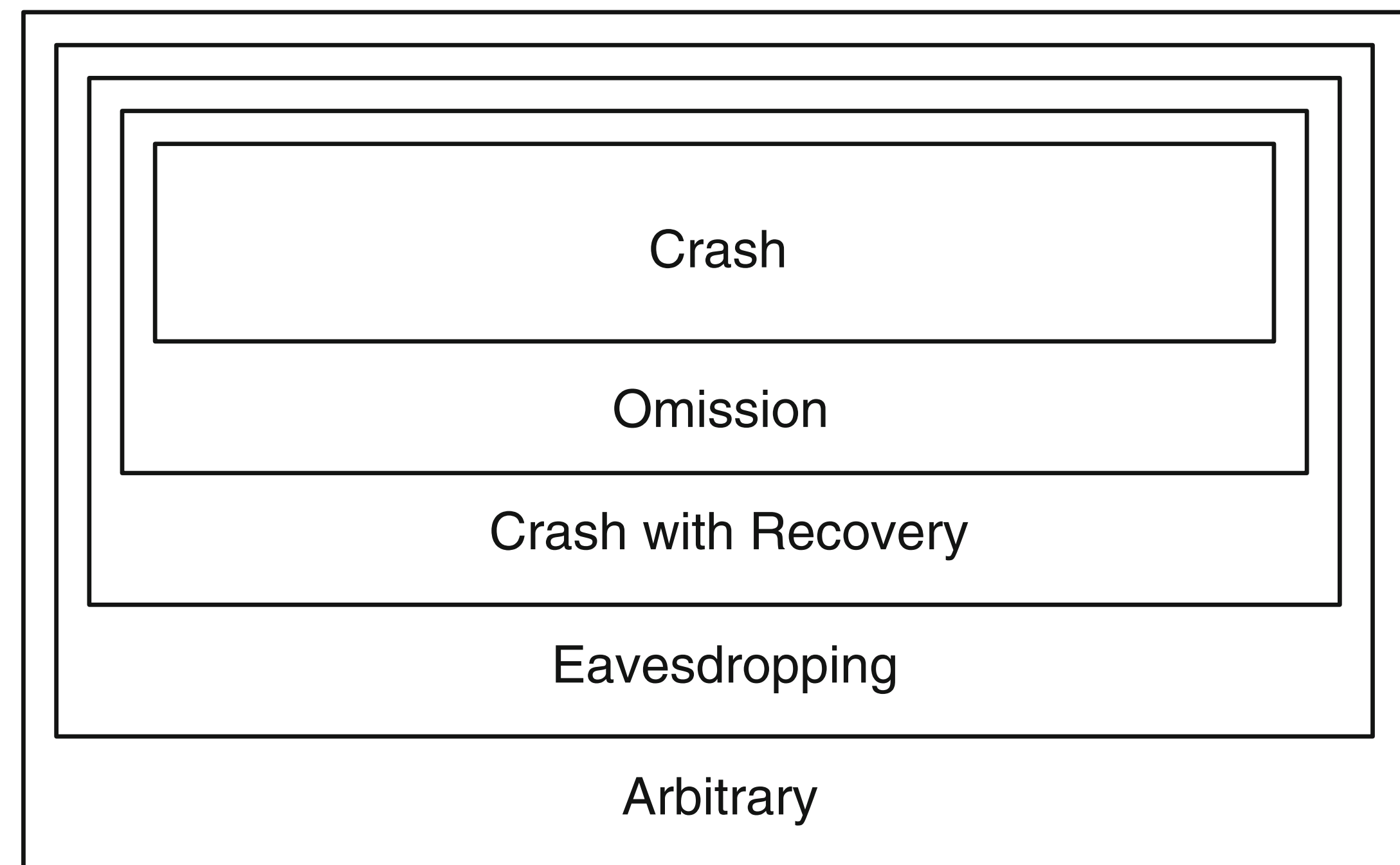


Figure 2.3: Types of process failures

Arbitrary Failures

Arbitrary Failures (Faults)

- **Definition:** *A process may behave in any conceivable way, including both accidental and malicious behavior*
- The most general type of failure

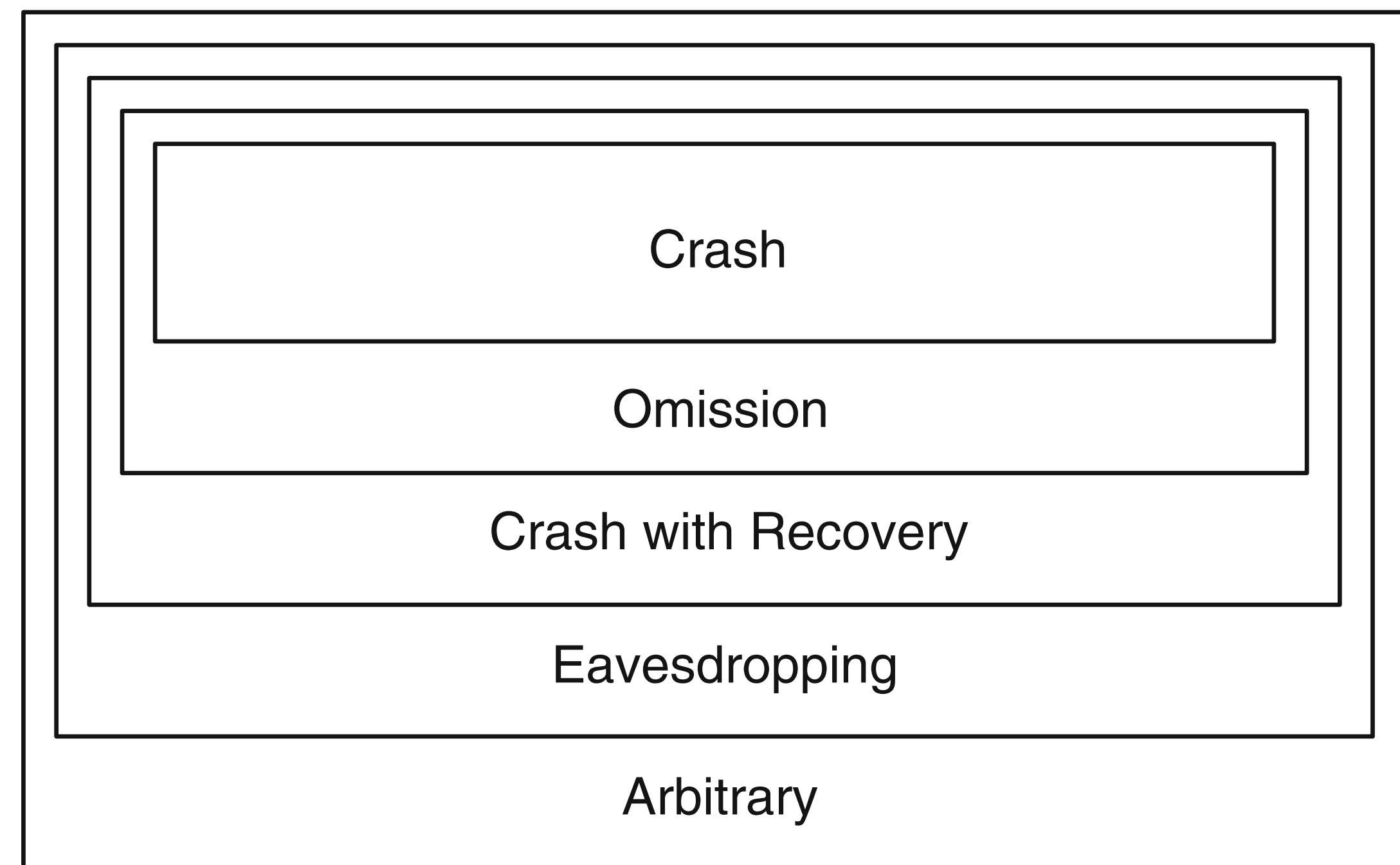


Figure 2.3: Types of process failures

Byzantine Failures

Arbitrary Failures (Faults): Byzantine Process Abstraction

- Process is **faulty** if it may
 - *deviate arbitrarily from its specification*
- Process is **correct** otherwise

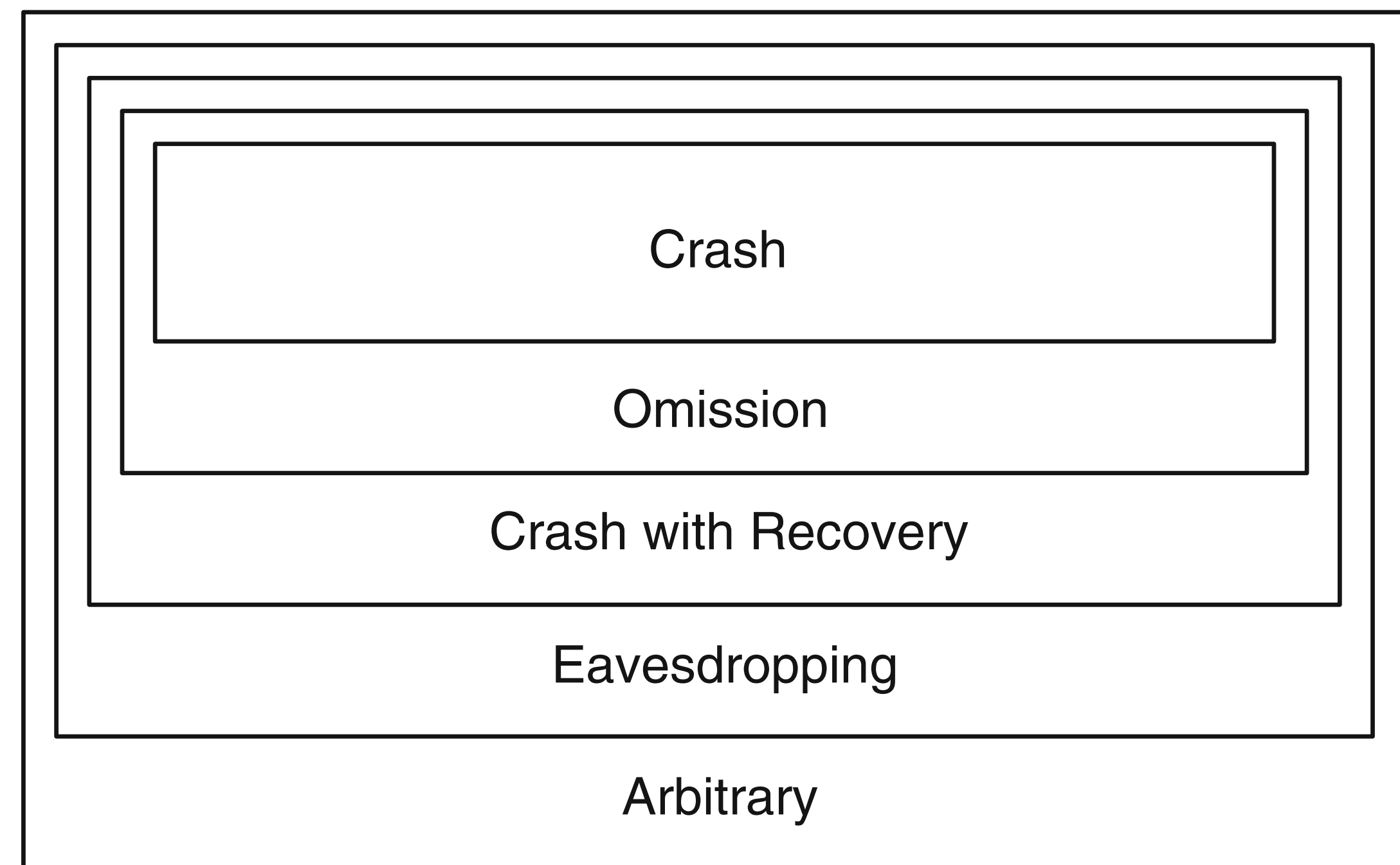


Figure 2.3: Types of process failures

Arbitrary Failures (Faults): Behavior of Byzantine Faulty Processes

- A faulty process may:
 - Generate any kind of output
 - Send arbitrary or inconsistent messages
- Often called:
 - Byzantine faulty or malicious processes

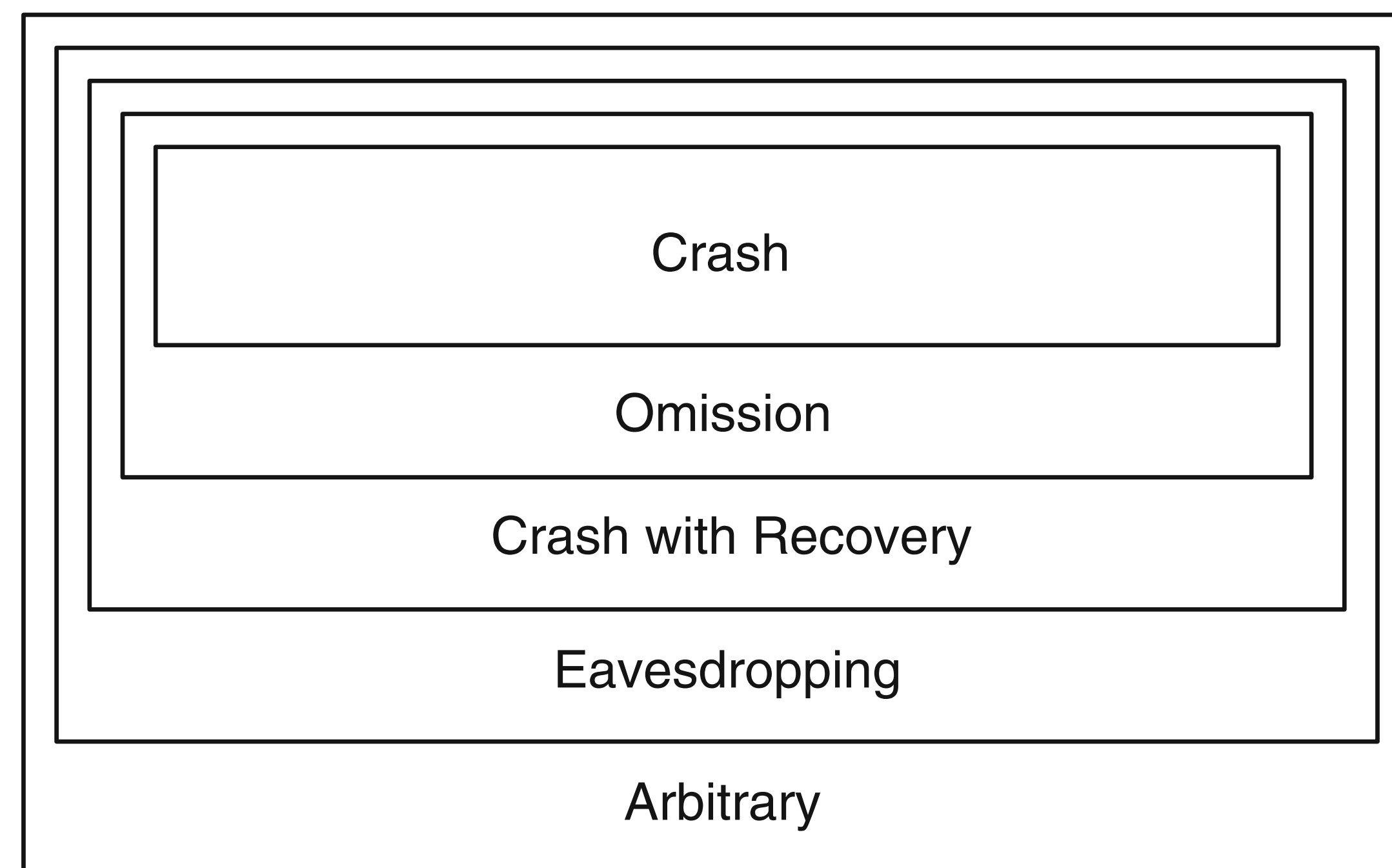


Figure 2.3: Types of process failures

Arbitrary Failures (Faults): Sources of Arbitrary Faults

- Malware (e.g., viruses or worms)
- Software bugs
- Programming language issues
- Compiler errors
- Runtime or middleware faults
- Operating system faults

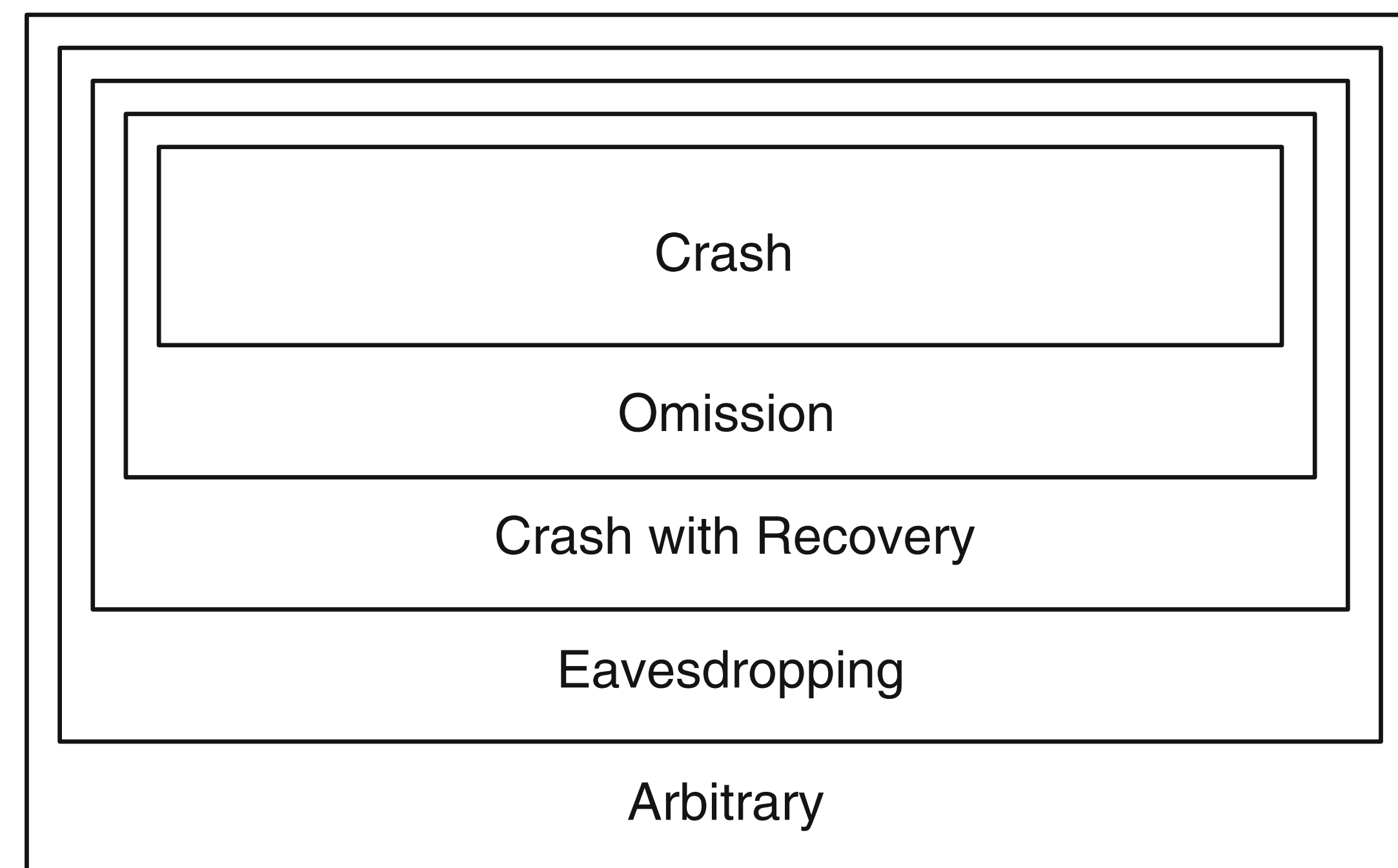


Figure 2.3: Types of process failures

Arbitrary Failures (Faults): Implications of Byzantine Fault Tolerance

- Systems that tolerate malicious faults also tolerate non-malicious faults
- Typically assume non-breakable cryptographic primitives
- Byzantine fault tolerance is costly in terms of:
 - Computation
 - Communication
 - Complexity

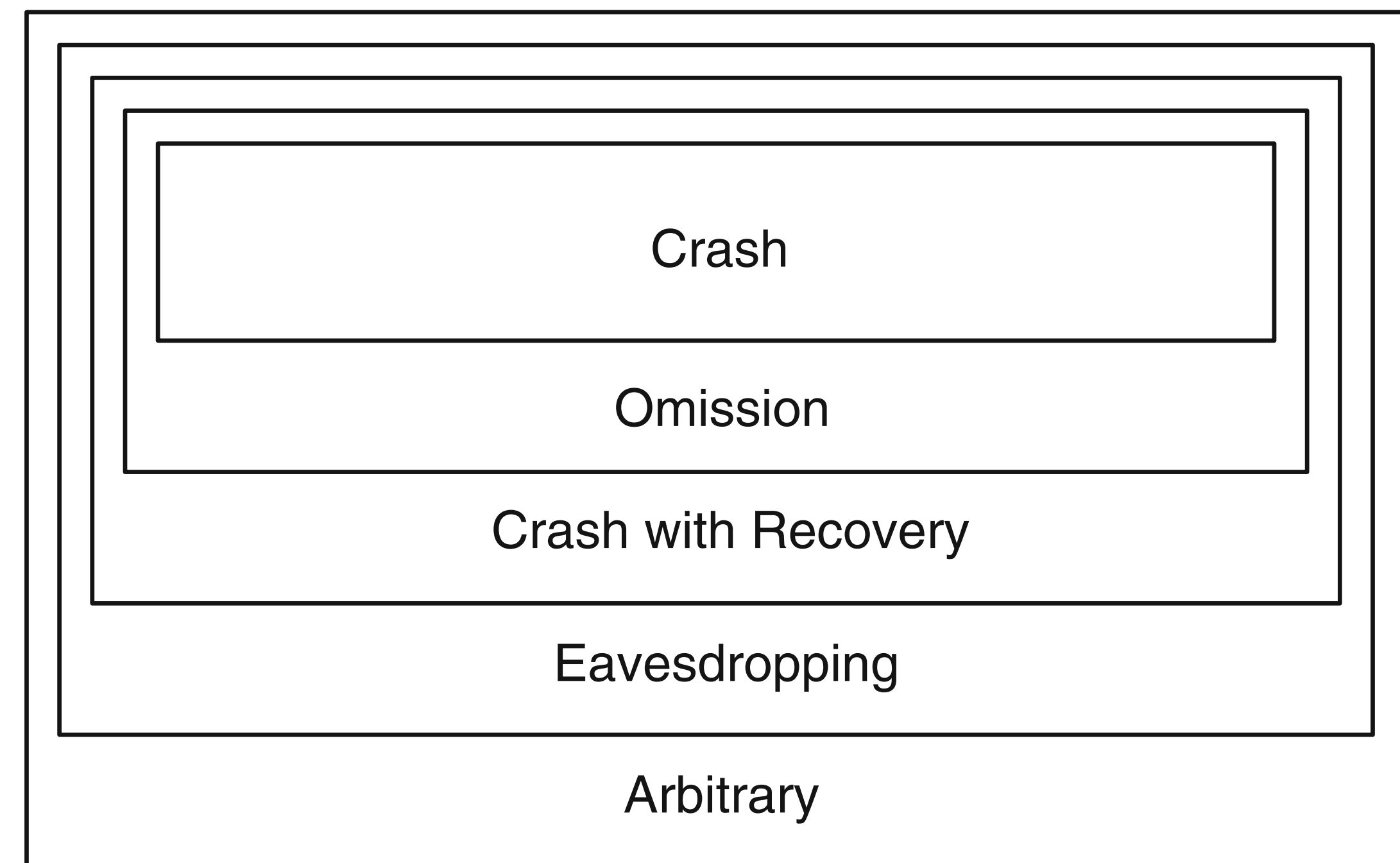


Figure 2.3: Types of process failures

Byzantine Generals

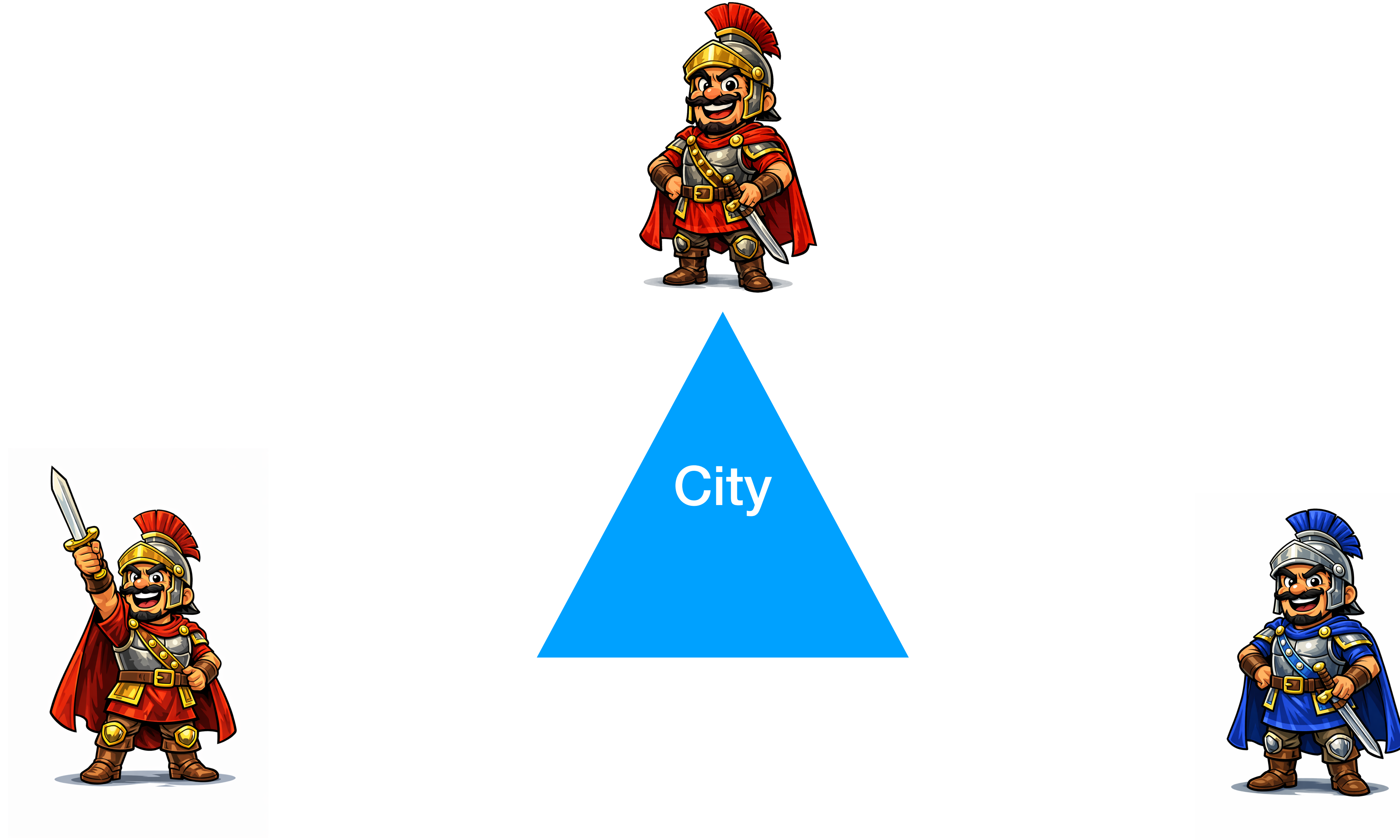
Arbitrary Failures (Faults)

- A *classic thought experiment* illustrating Byzantine failures
- Introduced to reason about *agreement in the presence of arbitrary failures*
- Models failures where components may
 - lie,
 - send conflicting messages, or
 - collude

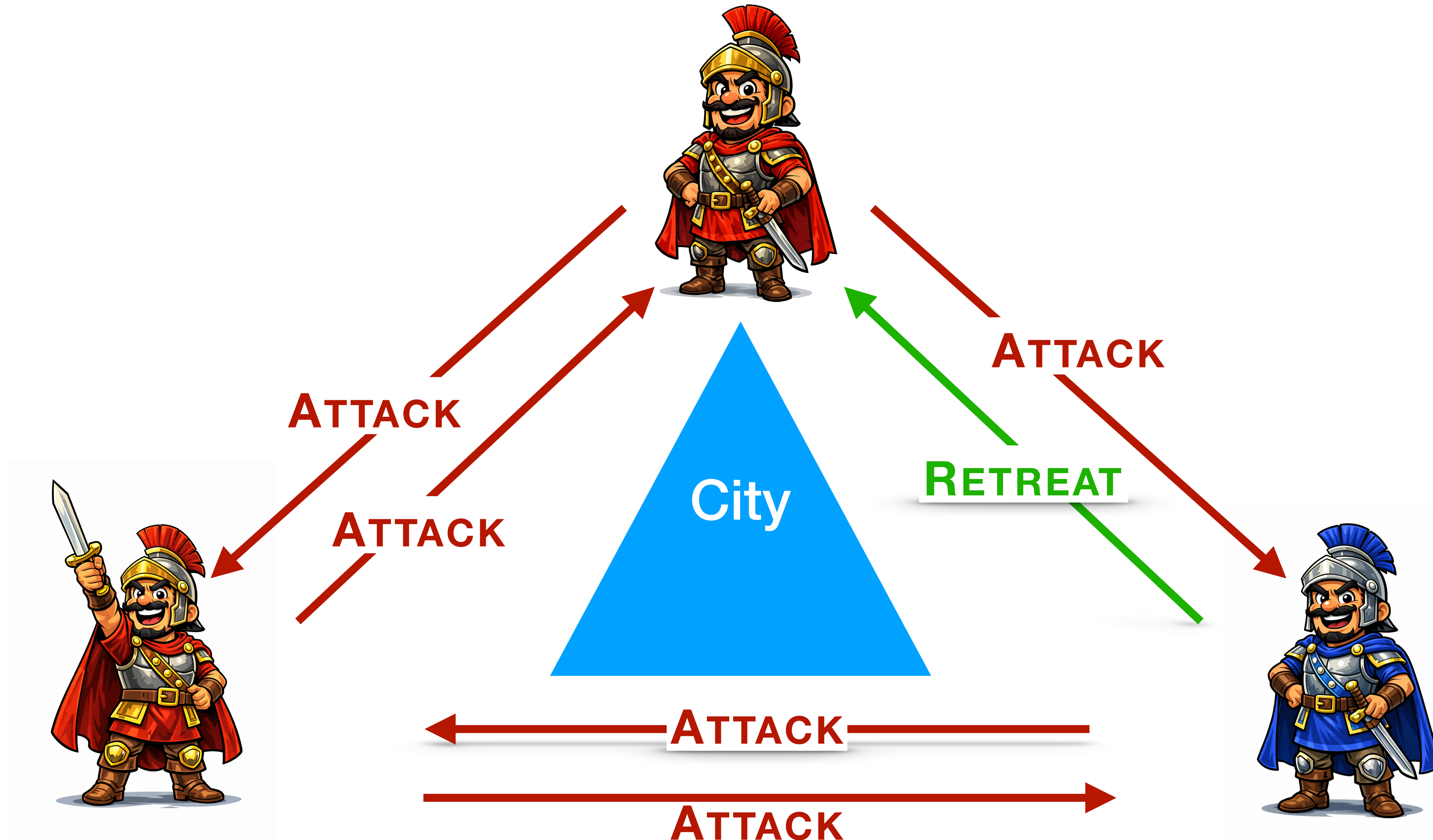
Arbitrary Failures (Faults): Problem Setting

- Several generals surround a city and must coordinate (attack or retreat)
- Communication is only via messengers
- Some generals may be Byzantine:
 - Send false or inconsistent messages
 - Try to prevent agreement
- **Goal:** *All correct generals must reach the same decision*

The Byzantine Generals Problem



The Byzantine Generals Problem



Arbitrary Failures (Faults): Key Insight and Implications

- Agreement is possible only if:
 - The number of faulty generals is bounded
 - The protocol tolerates arbitrary behavior
- Implications: Captures the essence of **Byzantine Fault Tolerance** (BFT)
- Shows why BFT protocols are:
 - More complex
 - More expensive than crash-fault-tolerant protocols

Joke time!

All abstractions are wrong,
but some are publishable

A process is just a machine,
with all the inconvenient
details abstracted away

In theory, the process is the unit of failure. In practice, it's whatever broke five minutes ago.

The process runs correctly...

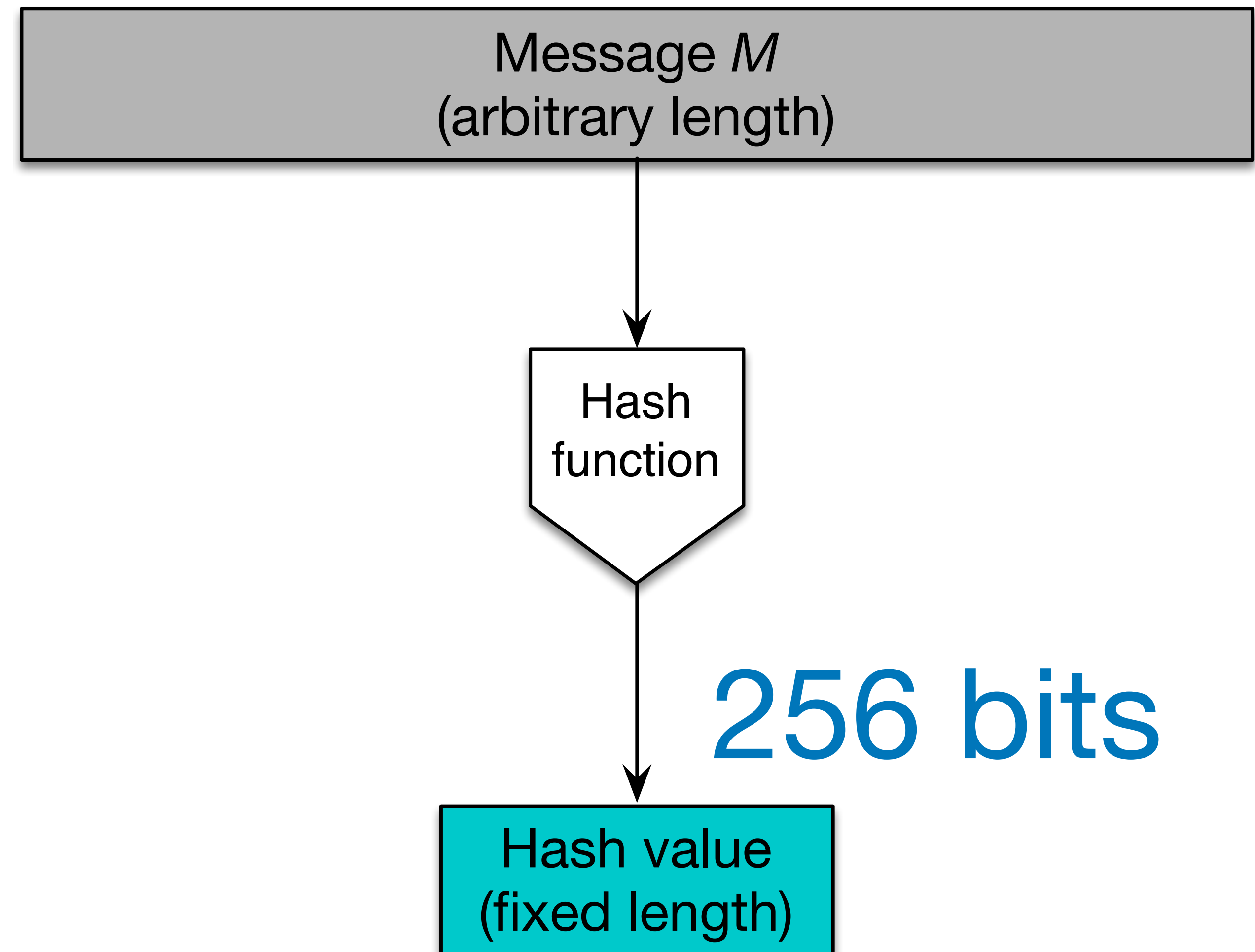
until it doesn't.

And then it politely stops existing.

Cryptographic Abstractions

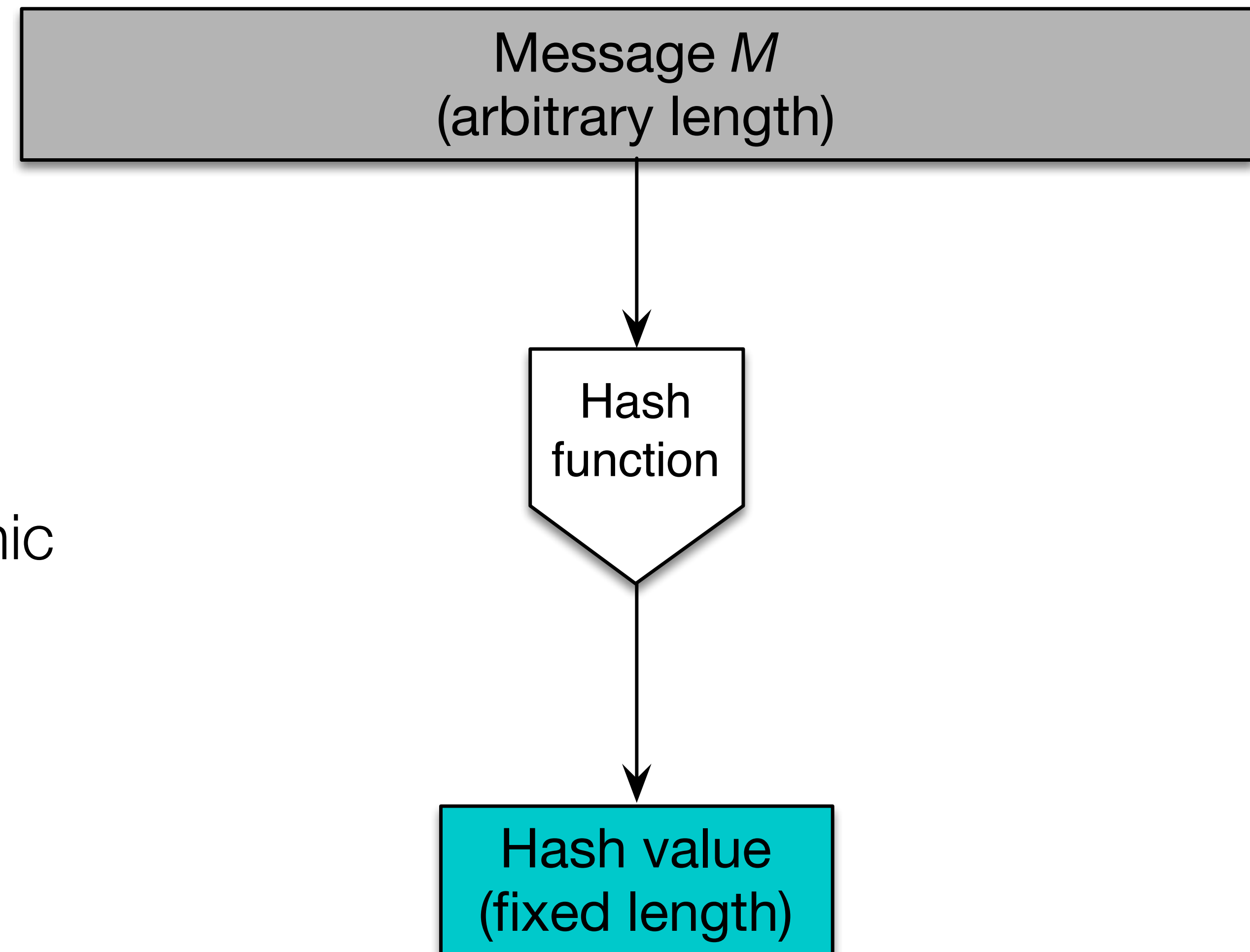
Cryptographic Hash Functions

- Function H maps:
 - Input of any length to
 - a short, *fixed-size* **tag**
- Deterministic:
 - same input $\Rightarrow$ same hash
- Efficient to compute



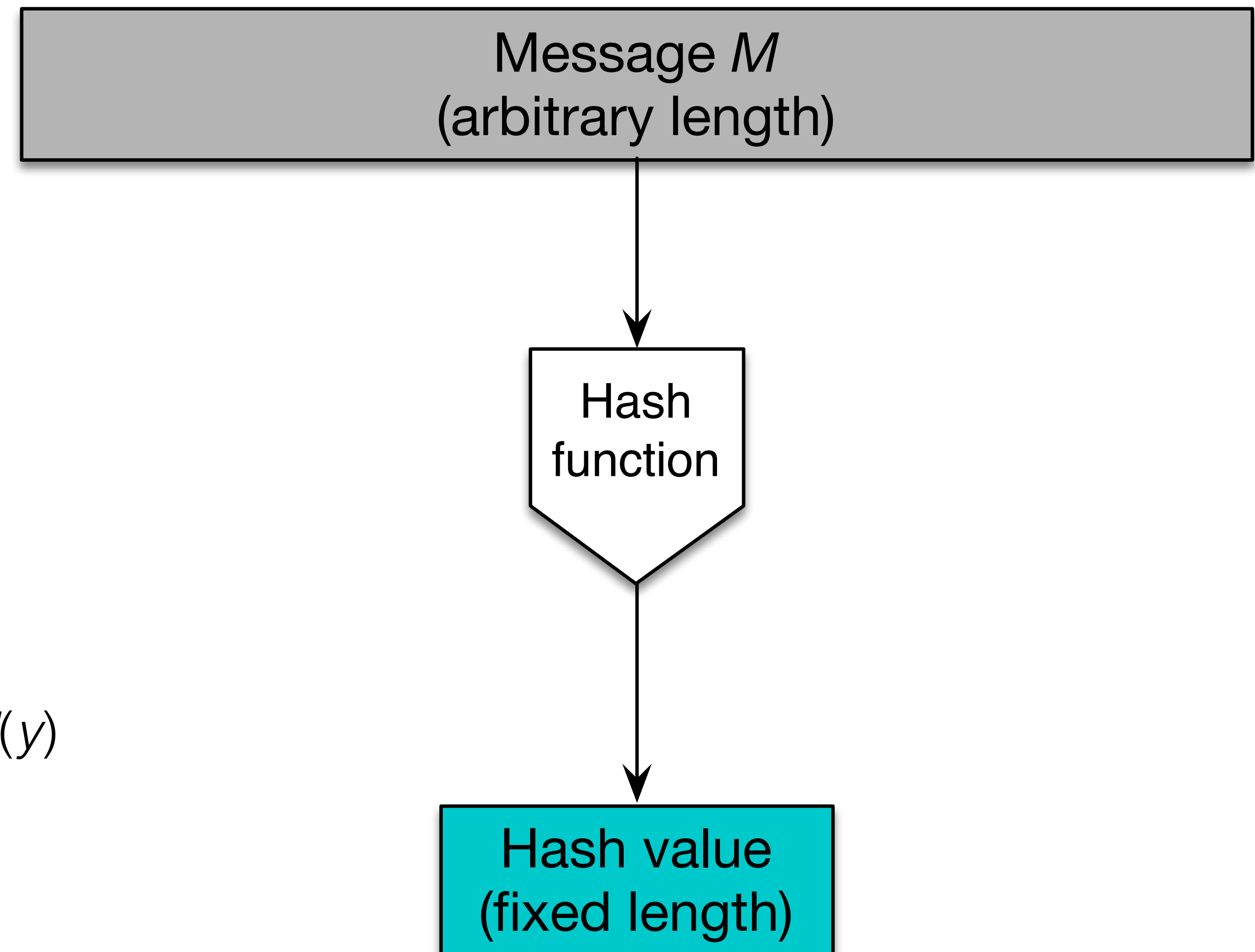
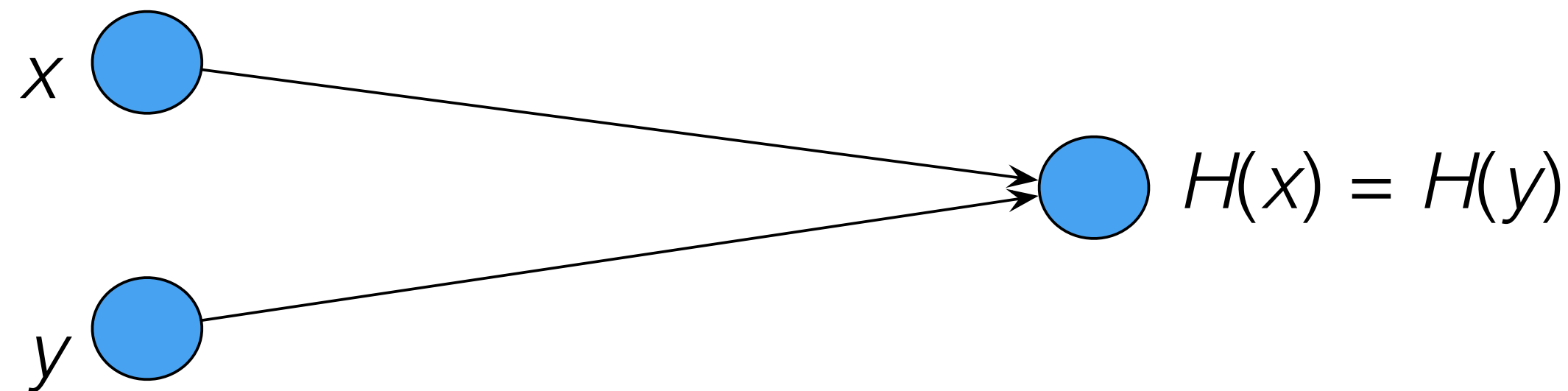
Cryptographic Hash Functions

- Typical Usages
 - Data integrity
 - Fingerprints: Digital digests
 - Building block for other cryptographic mechanisms

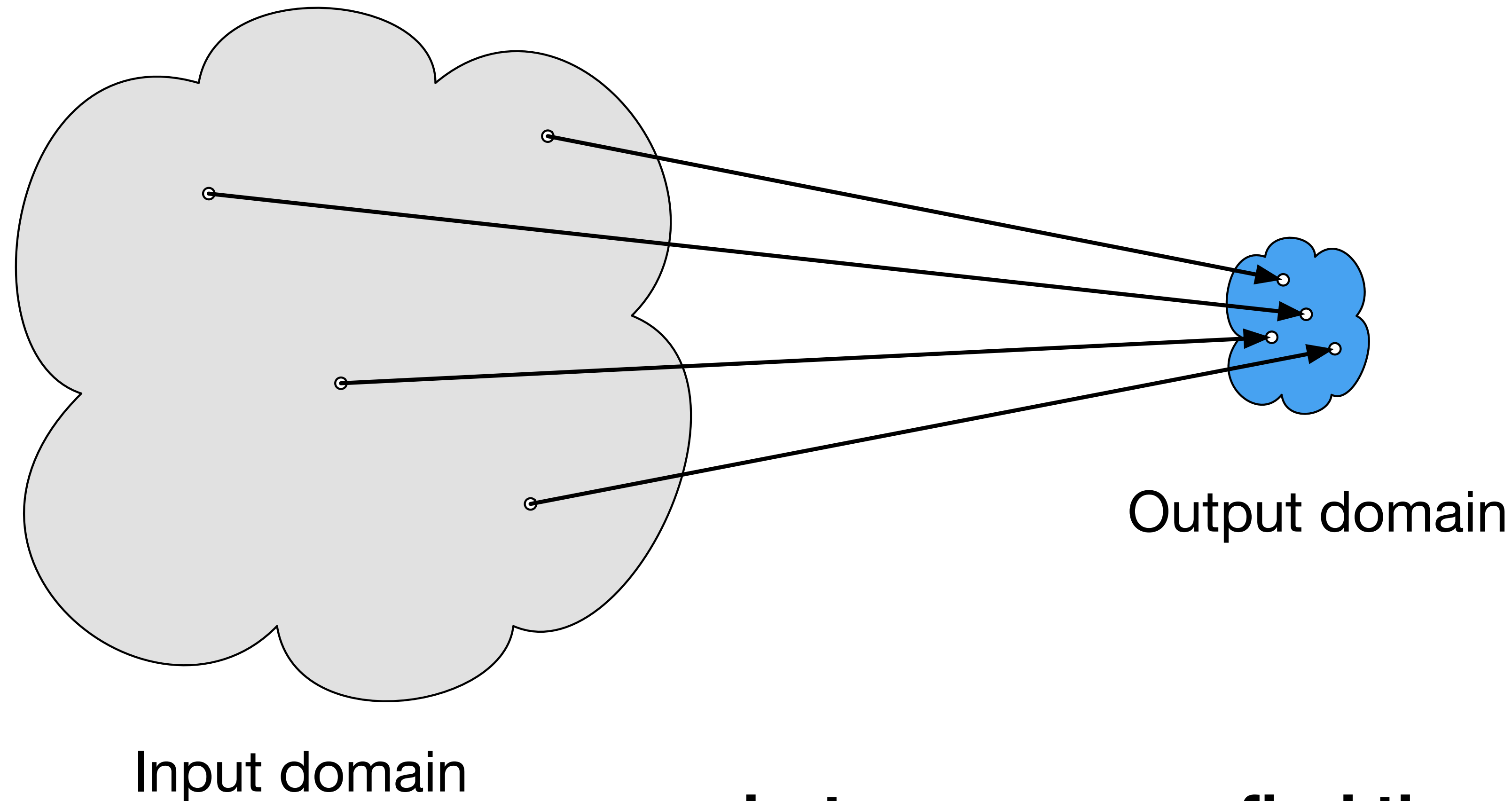


Cryptographic Hash Functions

- Collision-Freedom
 - Difficult to find two distinct values
 - $x \neq y$ such that $H(x) = H(y)$



Cryptographic Hash Functions: Collisions must exist...



... but can anyone find them?

Cryptographic Hash Functions



EASY



HARD



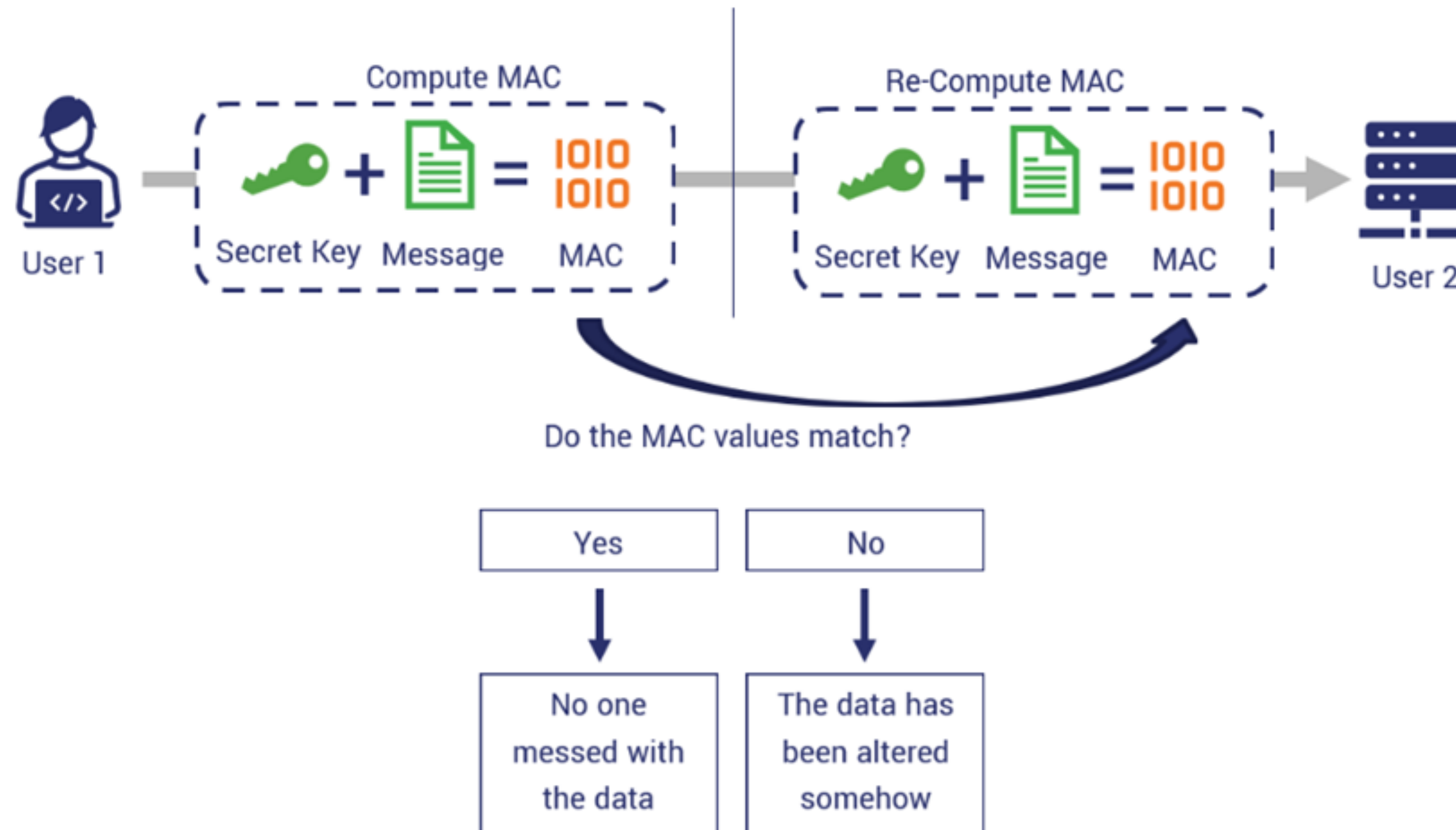
Message Authentication Codes (MACs)

- Purpose: Authenticates data between two processes
- API
 - **a** = authenticate(p, q, m)
 - **b** = verifyauth(q, p, m, **a**)
- Implementation
 - Symmetric cryptography
 - Shared secret key

Message Authentication Codes (MACs)

- Built from:
 - Hash functions
 - Stream or block ciphers
- Fast algorithms
- Limitation
 - Both parties can generate valid authenticators
 - **!** Cannot prove which party created the message

How Message Authentication Codes Work



Digital Signatures

- Purpose: Authenticates data with public verification
- API
 - $\mathbf{s} = \text{sign}(p, m)$
 - $\mathbf{b} = \text{verifysign}(q, m, \mathbf{s})$
- Key Properties
 - Only process p can create signatures (p represents the private key)
 - Anyone can verify (q represents the public key)

Digital Signatures

- Implementation
 - Asymmetric (public-key) cryptography
- Examples:
 - RSA
 - Elliptic Curve Cryptography

Digital Signatures: Why Digital Signatures Matter

- Stronger Guarantees than MACs
 - Authenticity can be relayed by an untrusted process
 - Enables non-repudiation
- Key Capability
 - Can prove to a third party
 - Who sent message m
- Typical Uses: Software updates, Certificates and PKI
- Byzantine fault-tolerant protocols
- Blockchains and distributed ledgers

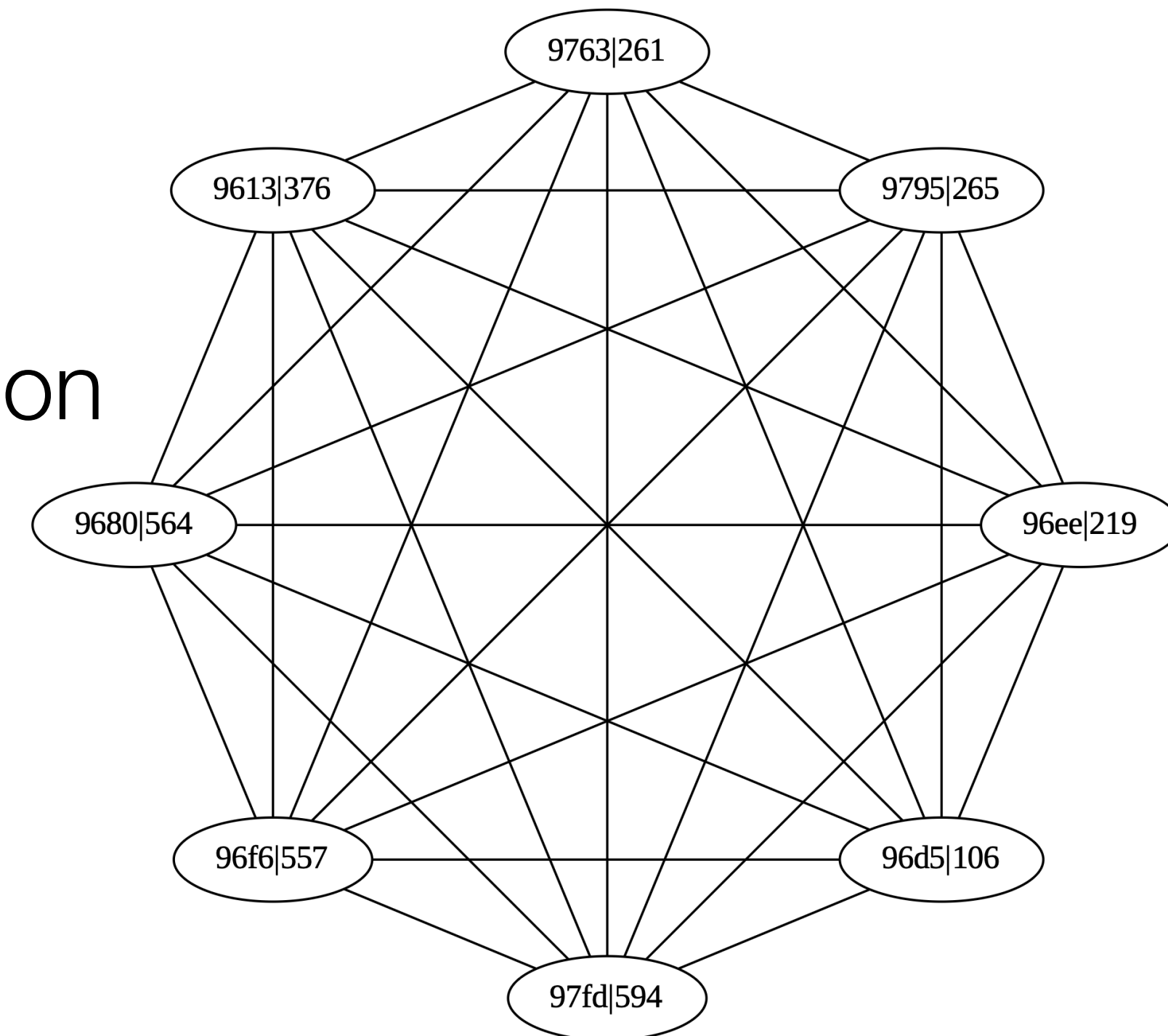
Joke time!

Add Joke crypto here!

Abstracting Communication

Link Abstraction

- *Every pair of processes is connected by a **bidirectional link***
 - Provides **full logical connectivity** among processes
- Independent of physical topology
 - Different network topologies can provide this abstraction
 - Often relies on routing algorithms
- Think of it as an overlay network
- Not the same as the Internet/ISO link layer



Why Link Abstractions?

- Simplifies distributed algorithms
- Hides low-level network complexity
- Allows reasoning at the process-to-process level
- Failures and recovery are handled below the algorithm layer

Link Failures

- Messages may be **lost**
- Messages may be **delayed** or **reordered**
- The network may be **attacked**
 - Inspect message contents
 - Modify messages
 - Prevent delivery
- Communication can be protected using **cryptography**

Fair-Loss Link Abstraction

- Messages **may be lost**
- Messages are **not systematically dropped**
 - The probability of successful delivery is **non-zero**
- No guarantees on delivery time or duplicates
- Comparable to a **physical link layer** abstraction

Stubborn Link Abstraction

- Sender **retransmits messages indefinitely**
- Retransmission mechanism is **hidden from higher layers**
- Guarantees:
 - If sender and receiver are correct, the message is sent **infinitely often**
- Does **not** suppress duplicates

Perfect (Reliable) Link Abstraction

- Ensures:
 - **No message loss** between correct processes
 - **No duplicate delivery**
- Adds mechanisms to:
 - Detect and suppress duplicates
- Often called **reliable links**
 - Similar to TCP semantics

Masking Link Failures

- Message loss: masked by **retransmission**
- Duplicate messages: masked by **duplicate detection**
- Link failures/partitions: masked by **routing**
- *Correctness holds as long as the underlying link does **not** cause an unbounded number of omissions*

Module 2.1: Interface and properties of fair-loss point-to-point links

Module:

Name: FairLossPointToPointLinks, **instance** fl .

Events:

Request: $\langle fl, Send \mid q, m \rangle$: Requests to send message m to process q .

Indication: $\langle fl, Deliver \mid p, m \rangle$: Delivers message m sent by process p .

Properties:

FLL1: Fair-loss: If a correct process p infinitely often sends a message m to a correct process q , then q delivers m an infinite number of times.

FLL2: Finite duplication: If a correct process p sends a message m a finite number of times to process q , then m cannot be delivered an infinite number of times by q .

FLL3: No creation: If some process q delivers a message m with sender p , then m was previously sent to q by process p .

Module 2.2: Interface and properties of stubborn point-to-point links

Module:

Name: StubbornPointToPointLinks, **instance** sl .

Events:

Request: $\langle sl, Send \mid q, m \rangle$: Requests to send message m to process q .

Indication: $\langle sl, Deliver \mid p, m \rangle$: Delivers message m sent by process p .

Properties:

SL1: Stubborn delivery: If a correct process p sends a message m once to a correct process q , then q delivers m an infinite number of times.

SL2: No creation: If some process q delivers a message m with sender p , then m was previously sent to q by process p .

Module 2.3: Interface and properties of perfect point-to-point links

Module:

Name: PerfectPointToPointLinks, **instance** pl .

Events:

Request: $\langle pl, Send \mid q, m \rangle$: Requests to send message m to process q .

Indication: $\langle pl, Deliver \mid p, m \rangle$: Delivers message m sent by process p .

Properties:

PL1: *Reliable delivery*: If a correct process p sends a message m to a correct process q , then q eventually delivers m .

PL2: *No duplication*: No message is delivered by a process more than once.

PL3: *No creation*: If some process q delivers a message m with sender p , then m was previously sent to q by process p .

Algorithm 2.1: Retransmit Forever

Implements:

StubbornPointToPointLinks, **instance** *sl*.

Uses:

FairLossPointToPointLinks, **instance** *fl*.

upon event $\langle sl, Init \rangle$ **do**

sent := $\emptyset$;

starttimer(Δ);

upon event $\langle Timeout \rangle$ **do**

forall $(q, m) \in sent$ **do**

trigger $\langle fl, Send \mid q, m \rangle$;

starttimer(Δ);

upon event $\langle sl, Send \mid q, m \rangle$ **do**

trigger $\langle fl, Send \mid q, m \rangle$;

sent := *sent* $\cup \{(q, m)\}$;

upon event $\langle fl, Deliver \mid p, m \rangle$ **do**

trigger $\langle sl, Deliver \mid p, m \rangle$;

Algorithm 2.2: Eliminate Duplicates

Implements:

PerfectPointToPointLinks, **instance** *pl*.

Uses:

StubbornPointToPointLinks, **instance** *sl*.

upon event $\langle pl, Init \rangle$ **do**

delivered := $\emptyset$;

upon event $\langle pl, Send \mid q, m \rangle$ **do**

trigger $\langle sl, Send \mid q, m \rangle$;

upon event $\langle sl, Deliver \mid p, m \rangle$ **do**

if $m \notin delivered$ **then**

delivered := *delivered* $\cup \{m\}$;

trigger $\langle pl, Deliver \mid p, m \rangle$;

Authenticated Perfect Links

- In the **Byzantine process abstraction**, links *may behave arbitrarily*
 - Messages can be lost, modified, delayed, or forged
 - DoS attacks may prevent communication
- Assumption:
 - *An adversary cannot block all communication between correct processes*

Authenticated Perfect Links

- Provide:
 - **Reliable delivery** between correct processes
 - **No message forgery**
- Guarantees
 - *A message received from a **correct sender** was actually sent by that sender*
- Implemented using cryptographic authentication (MACs or digital signatures)
- Fundamental abstraction for Byzantine fault-tolerant protocols

Module 2.5: Interface and properties of authenticated perfect point-to-point links

Module:

Name: AuthPerfectPointToPointLinks, **instance** al .

Events:

Request: $\langle al, Send \mid q, m \rangle$: Requests to send message m to process q .

Indication: $\langle al, Deliver \mid p, m \rangle$: Delivers message m sent by process p .

Properties:

AL1: Reliable delivery: If a correct process sends a message m to a correct process q , then q eventually delivers m .

AL2: No duplication: No message is delivered by a correct process more than once.

AL3: Authenticity: If some correct process q delivers a message m with sender p and process p is correct, then m was previously sent to q by p .

Algorithm 2.4: Authenticate and Filter

Implements:

AuthPerfectPointToPointLinks, **instance** *al*.

Uses:

StubbornPointToPointLinks, **instance** *sl*.

upon event $\langle al, Init \rangle$ **do**

$delivered := \emptyset;$

upon event $\langle al, Send \mid q, m \rangle$ **do**

$a := authenticate(self, q, m);$

trigger $\langle sl, Send \mid q, [m, a] \rangle;$

upon event $\langle sl, Deliver \mid p, [m, a] \rangle$ **do**

if $verifyauth(self, p, m, a) \wedge m \notin delivered$ **then**

$delivered := delivered \cup \{m\};$

trigger $\langle al, Deliver \mid p, m \rangle;$

Joke time!

Physical layer:
Packets get lost

Link abstraction:

No, they don't

Algorithm:
Great, let's prove
correctness

Perfect links are what you get when you add enough retransmission and stop asking uncomfortable questions.

Timing Assumptions

Timing Models in Distributed Systems

Combining Process and Link abstractions into Timing Models

- Distributed systems are fundamentally about time
- Key questions:
 - Can we bound **process execution speed**?
 - Can we bound **message transmission delays**?
 - Can we rely on **clocks**?
- ➡ The answers define the **system model**

Asynchronous System Model

Asynchronous System Model

- **No timing assumptions**
- **No bounds** on:
 - Process execution speed
 - Message transmission delay
 - Clock drift
- ➡ Time cannot be used for coordination

Asynchronous System Model: Logical Time

- Even without physical time:
 - Can define **logical time**
 - Can reason about **causal ordering**
- Key idea:
 - **Happened-before** relation

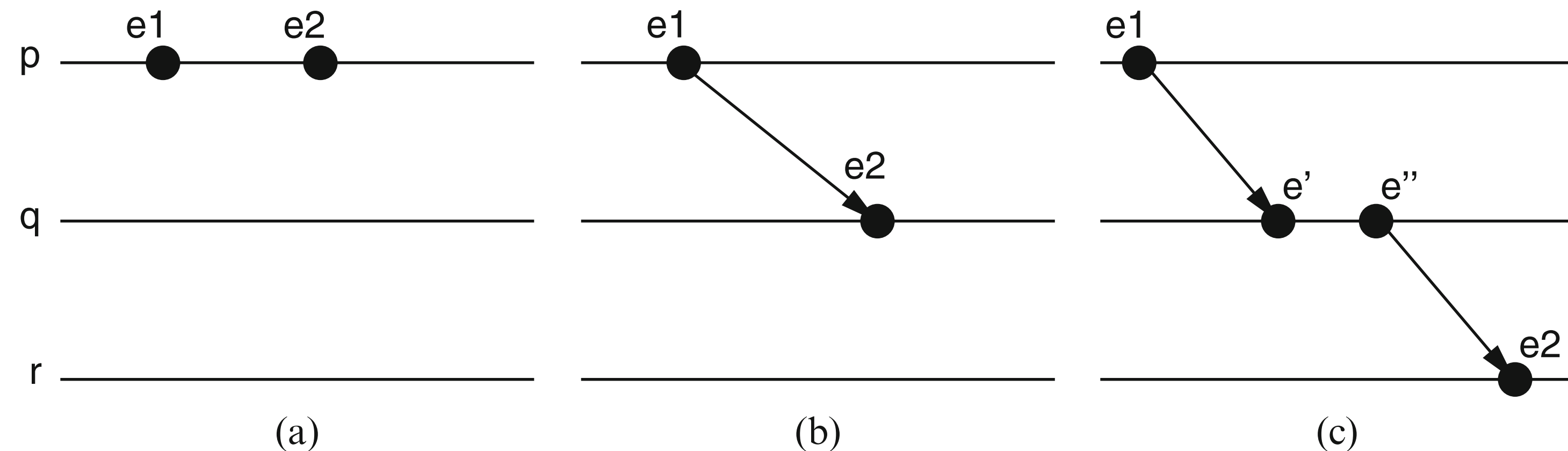


Figure 2.5: The *happened-before* relation

Asynchronous System Model: Logical Time

- Used for:
 - Debugging
 - Snapshots
 - Reasoning about consistency

Asynchronous System Model: Limits of Asynchrony

- Some abstractions **cannot** be implemented asynchronously
- Most famous example:
 - **Consensus is impossible** in a purely *asynchronous system with failures*
- Known as:
 - FLP impossibility result
- ➡ No algorithm can guarantee termination

Synchronous System Model

Synchronous System Model

- **Strong timing assumptions**
- *Known upper bounds on:*
 - Computation (process execution time)
 - Communication (message delays)
 - Clock drift (physical clocks stay close)
-  Time becomes a usable coordination mechanism

Synchronous System Model: Synchrony Enables

- **Timed failure detection**
 - Crashes detected within a bounded time
- **Time-based coordination**
 - Leases (temporary *exclusive access* rights)
- **Synchronized clocks**, e.g., NTP
- **Worst-case performance guarantees**
 - Precise bounds on latency and termination

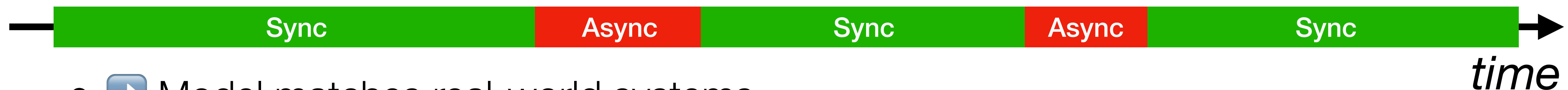
Synchronous System Model: Why Full Synchrony Is Unrealistic

- **Problem:** hard to guarantee timing bounds with high probability
- Violations caused by:
 - Network congestion
 - Message loss → retransmissions
 - OS scheduling delays
 - Memory pressure (paging/swapping)
-  Real systems *sometimes* behave asynchronously

Partially Synchronous System Model

Partially Synchronous System Model

- Middle ground between sync and async
- Timing bounds:
 - Hold **most of the time**
 - Occasionally violated
- System alternates between **Asynchronous periods** and **Synchronous periods**



- ➡ Model matches real-world systems

Partially Synchronous System Model: Design Principle

- **Always safe**
 - Even during asynchronous periods
- **As live as possible**
 - Progress when timing assumptions hold
-  Safety may still be violated if:
 - Failure assumptions are violated:  More than f failures

Partially Synchronous System Model: Liveness vs Safety Tension

- Example: Retransmissions ensure **reliability** (safety)
 - But causes **unpredictable delays**
 - Can violate time bounds (liveness)
- ➡ **Fundamental tension:**
 - Improving safety often hurts liveness

Partially Synchronous System Model: Liveness vs Safety Tension

- **Key assumption:**
 - Timing bounds hold **eventually**
- Meaning:
 - There exist synchronous periods
 - Some are long enough for the algorithm to terminate
- ➡ Foundation for:
 - Practical consensus algorithms
 - Real-world fault-tolerant systems

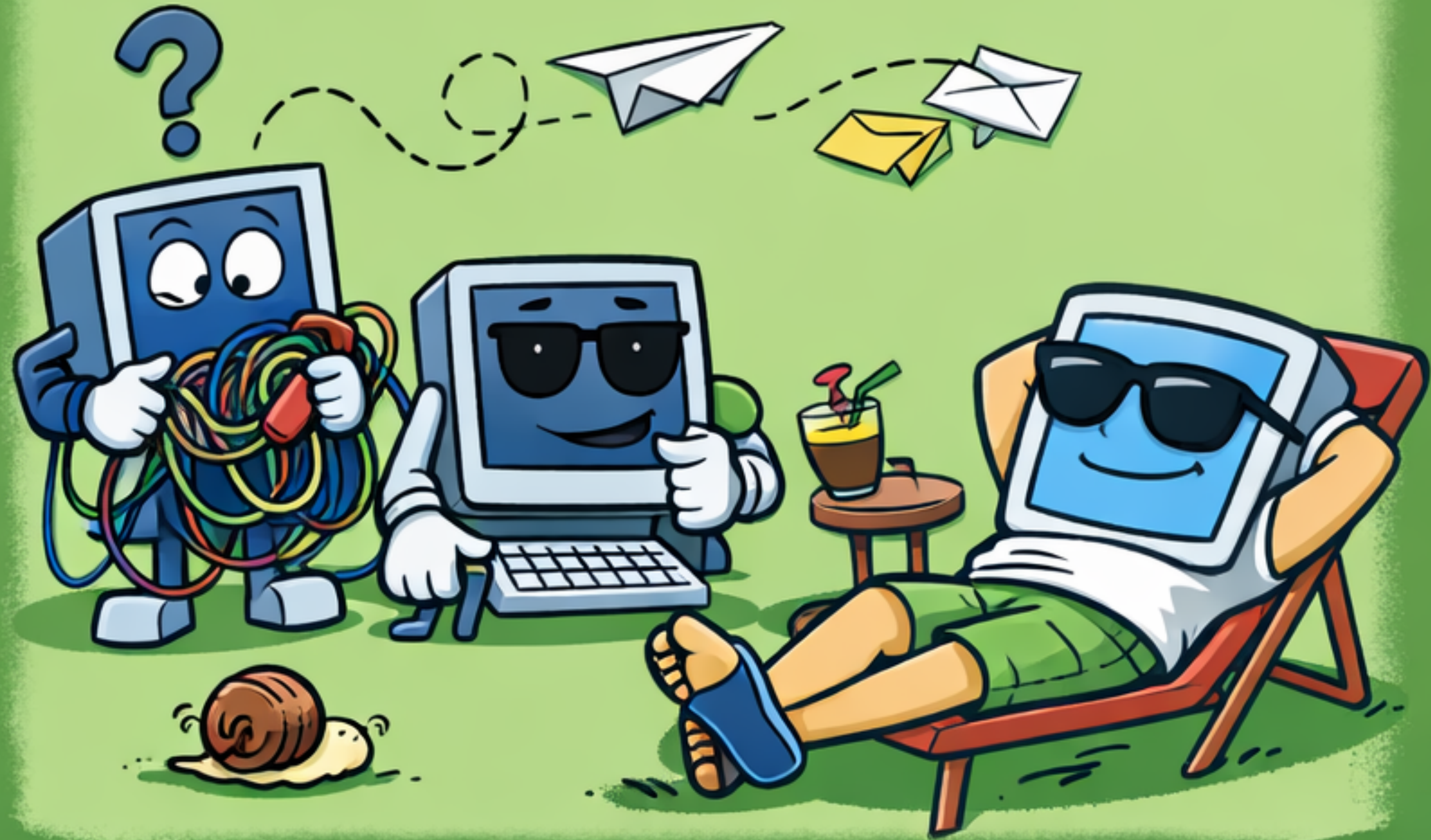
Joke time!

Synchronous



We're all just waiting for Bob..."

Asynchronous



Bob will get back to us... maybe."

Synchronous:
“Failure is detectable.”

Synchronous:
“Nothing is wrong.
We’re just waiting.”

Asynchronous:
“Failure is a rumor.”

Asynchronous:
“Time has no meaning
here.”

Abstracting Time

Asynchronous System Model: No Timing Assumptions

- *Only process and link abstractions*
- These abstractions are **insufficient** for:
 - Synchronous systems
 - Partially synchronous systems
- ➡ We need time-related abstractions

Synchronous and Partially Synchronous Models: We Need Time

- Augment process and link abstractions with **timing capabilities**
- Introduce failure detectors
 - Rely on **timing assumptions**

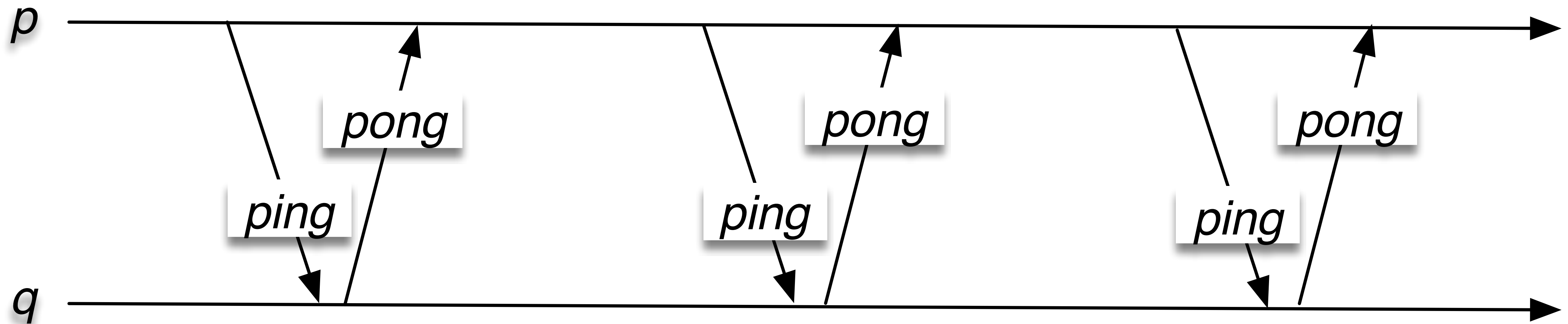
Failure Detection (Synchronous Model)

Failure Detector (FD)

- A distributed oracle that provides information about **crashed processes**
- Key observations:
 - Implemented using *timeouts*
 - Failures **cannot** be detected **perfectly**
 - Suspicions may be: **Accurate** or **Inaccurate**

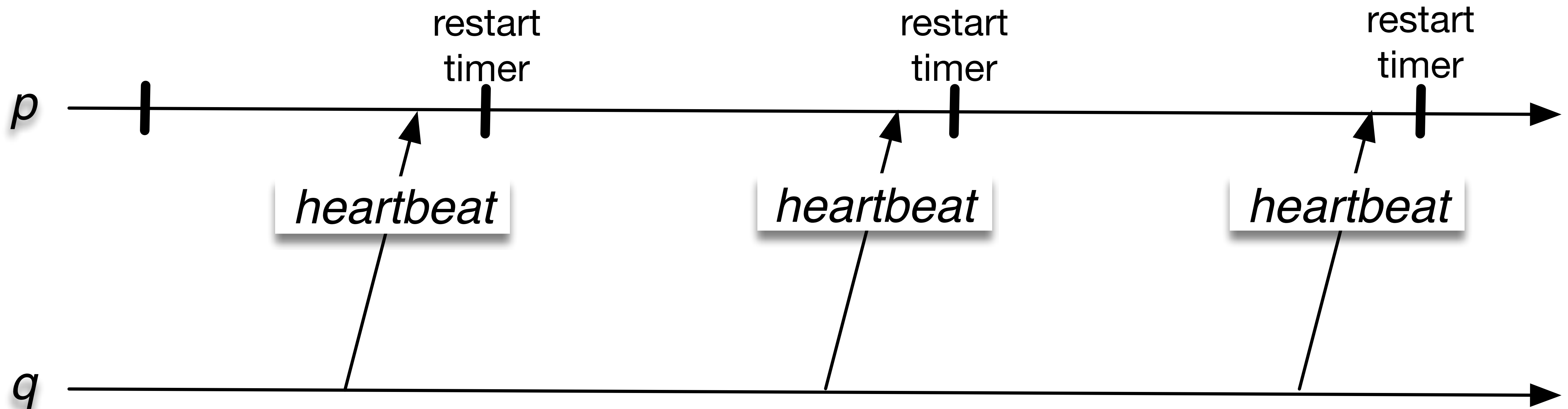
Two Main Approaches to Failure Detection

- Periodic **Ping**
 - Request $\rightarrow$ await reply



Two Main Approaches to Failure Detection

- Periodic **Heartbeat**
 - Expect heartbeat within a timeout
 - React locally on timeout



Perfect Failure Detector (P)

- Requires a **synchronous** system model
- ➡ Only possible with *known bounds* on:
 - Message delay
 - Processing time

Module 2.6: Interface and properties of the perfect failure detector

Module:

Name: PerfectFailureDetector, **instance** $\mathcal{P}$.

Events:

Indication: $\langle \mathcal{P}, \text{Crash} \mid p \rangle$: Detects that process p has crashed.

Properties:

PFD1: *Strong completeness:* Eventually, every process that crashes is permanently detected by every correct process.

PFD2: *Strong accuracy:* If a process p is detected by any process, then p has crashed.

Algorithm 2.5: Exclude on Timeout

Implements:

PerfectFailureDetector, **instance** $\mathcal{P}$.

Uses:

PerfectPointToPointLinks, **instance** pl .

upon event $\langle \mathcal{P}, \text{Init} \rangle$ **do**

$alive := \Pi$;

$detected := \emptyset$;

$starttimer(\Delta)$;

upon event $\langle \text{Timeout} \rangle$ **do**

forall $p \in \Pi$ **do**

if $(p \notin alive) \wedge (p \notin detected)$ **then**

$detected := detected \cup \{p\}$;

trigger $\langle \mathcal{P}, \text{Crash} \mid p \rangle$;

trigger $\langle pl, \text{Send} \mid p, [\text{HEARTBEATREQUEST}] \rangle$;

$alive := \emptyset$;

$starttimer(\Delta)$;

upon event $\langle pl, \text{Deliver} \mid q, [\text{HEARTBEATREQUEST}] \rangle$ **do**

trigger $\langle pl, \text{Send} \mid q, [\text{HEARTBEATREPLY}] \rangle$;

upon event $\langle pl, \text{Deliver} \mid p, [\text{HEARTBEATREPLY}] \rangle$ **do**

$alive := alive \cup \{p\}$;

Performance Tradeoffs: Timeout Selection

- **Short** timeout
 - Fast failure response
 - More *false suspicions*
- **Long** timeout
 - Slow failure response
 - Fewer false suspicions

Performance Optimization: Two-Level Failure Detection

- **Fast Suspicion** (short timeout)
- **Slow Exclusion** (longer timeout)
- Helps balance:
 - Responsiveness
 - Accuracy

Leader Election (Synchronous Model)

Leader Election

- Many distributed algorithms require a leader:
- A process that:
 - Has **not failed**
 - **Coordinates** the protocol
- Leader election can be built using failure detectors

Primary–Backup Replication (Diagram)

Module 2.7: Interface and properties of leader election

Module:

Name: LeaderElection, **instance** le .

Events:

Indication: $\langle le, Leader \mid p \rangle$: Indicates that process p is elected as leader.

Properties:

LE1: *Eventual detection*: Either there is no correct process, or some correct process is eventually elected as the leader.

LE2: *Accuracy*: If a process is leader, then all previously elected leaders have crashed.

Algorithm 2.6: Monarchical Leader Election

Implements:

LeaderElection, **instance** le .

Uses:

PerfectFailureDetector, **instance** $\mathcal{P}$.

upon event $\langle le, Init \rangle$ **do**

$suspected := \emptyset$;

$leader := \perp$;

upon event $\langle \mathcal{P}, Crash \mid p \rangle$ **do**

$suspected := suspected \cup \{p\}$;

upon $leader \neq \text{maxrank}(\Pi \setminus suspected)$ **do**

$leader := \text{maxrank}(\Pi \setminus suspected)$;

trigger $\langle le, Leader \mid leader \rangle$;

Leader Election: Selection by Rank

- Rank-based leader election
 - Each process has a **fixed rank**
 - E.g., IP address, port number
- Leader = process with **highest rank** *among non-suspected*

Leader Election: Selection by Rank

- $\text{maxrank}(S)$:
 - Process with highest rank in set S
- arg max :
 - Argument that maximizes a function

$$\text{maxrank}(S) = \text{arg max}_{p \in S} \{ \text{rank}(p) \}$$

Eventually Perfect Failure Detection (Partially Synchronous Model)

Eventually Perfect Failure Detection ($\diamond P$)

- Now, assume a **partially synchronous system**
- **Eventually, the system behaves synchronously**

Global Stabilization Time

- ➡ Timing guarantees eventually hold
- Initially, *unknown bounds* on:
 - Message delay
 - Processing time
- but exist after some *unknown* **Global Stabilization Time** (GST)

Global Stabilization Time: Implications

- Timeouts *may be violated before* GST
- Timeouts *become reliable after* GST
- Enables eventual properties (e.g., $\diamond P$, eventual leader election)

Eventually Perfect Failure Detection ($\diamond P$): Intuition

- Initially:
 - Timeouts may be too short
 - False suspicions occur
- When suspicion happens:
 - **Increase timeout**
- Eventually:
 - Timeout exceeds actual delays, **accuracy stabilizes**

Eventually Perfect Failure Detection ($\diamond P$): Practical Problem

- Issue
 - Timeout **only increases**
 - **Never decreases** when *synchrony returns*
- Leads to:
 - **Very slow detection over time**
- ➡ Real systems use:
 - Adaptive timeouts
 - Statistical methods

Module 2.8: Interface and properties of the eventually perfect failure detector

Module:

Name: EventuallyPerfectFailureDetector, **instance** $\diamond\mathcal{P}$.

Events:

Indication: $\langle \diamond\mathcal{P}, \text{Suspect} \mid p \rangle$: Notifies that process p is suspected to have crashed.

Indication: $\langle \diamond\mathcal{P}, \text{Restore} \mid p \rangle$: Notifies that process p is not suspected anymore.

Properties:

EPFD1: *Strong completeness:* Eventually, every process that crashes is permanently suspected by every correct process.

EPFD2: *Eventual strong accuracy:* Eventually, no correct process is suspected by any correct process.

Algorithm 2.7: Increasing Timeout

Implements:

 EventuallyPerfectFailureDetector, **instance** $\diamond\mathcal{P}$.

Uses:

 PerfectPointToPointLinks, **instance** pl .

upon event $\langle \diamond\mathcal{P}, \text{Init} \rangle$ **do**
 $alive := \Pi$;

 $suspected := \emptyset$;

 $delay := \Delta$;

 $starttimer(delay)$;

New process p is no longer in 'alive' set and should be 'suspected'.

New process p is now in 'alive' and was 'suspected'.

upon event $\langle \text{Timeout} \rangle$ **do**
if $alive \cap suspected \neq \emptyset$ **then**
 $delay := delay + \Delta$;

forall $p \in \Pi$ **do**
if $(p \notin alive) \wedge (p \notin suspected)$ **then**
 $suspected := suspected \cup \{p\}$;

trigger $\langle \diamond\mathcal{P}, \text{Suspect} \mid p \rangle$;

else if $(p \in alive) \wedge (p \in suspected)$ **then**
 $suspected := suspected \setminus \{p\}$;

trigger $\langle \diamond\mathcal{P}, \text{Restore} \mid p \rangle$;

trigger $\langle pl, \text{Send} \mid p, [\text{HEARTBEATREQUEST}] \rangle$;

 $alive := \emptyset$;

 $starttimer(delay)$;

Both 'alive' and 'suspected':
One or more processes were suspected incorrectly;
Update our timeout 'delay'.

upon event $\langle pl, \text{Deliver} \mid q, [\text{HEARTBEATREQUEST}] \rangle$ **do**
trigger $\langle pl, \text{Send} \mid q, [\text{HEARTBEATREPLY}] \rangle$;

upon event $\langle pl, \text{Deliver} \mid p, [\text{HEARTBEATREPLY}] \rangle$ **do**
 $alive := alive \cup \{p\}$;

Eventual Leader Detection (Partially Synchronous Model)

Eventual Leader Election: Using $\diamond P$

- Eventually:
 - All correct processes **trust the same leader**
 - Leader = highest-ranked non-suspected process
- Once accuracy stabilizes:
 - Leader remains stable

Module 2.9: Interface and properties of the eventual leader detector

Module:

Name: EventualLeaderDetector, **instance** Ω .

Events:

Indication: $\langle \Omega, Trust \mid p \rangle$: Indicates that process p is trusted to be leader.

Properties:

ELD1: *Eventual accuracy*: There is a time after which every correct process trusts some correct process.

ELD2: *Eventual agreement*: There is a time after which no two correct processes trust different correct processes.

Algorithm 2.8: Monarchical Eventual Leader Detection

Implements:

EventualLeaderDetector, **instance** Ω .

Uses:

EventuallyPerfectFailureDetector, **instance** $\diamond\mathcal{P}$.

upon event $\langle \Omega, \text{Init} \rangle$ **do**

$\text{suspected} := \emptyset;$

$\text{leader} := \perp;$

upon event $\langle \diamond\mathcal{P}, \text{Suspect} \mid p \rangle$ **do**

$\text{suspected} := \text{suspected} \cup \{p\};$

upon event $\langle \diamond\mathcal{P}, \text{Restore} \mid p \rangle$ **do**

$\text{suspected} := \text{suspected} \setminus \{p\};$

upon $\text{leader} \neq \text{maxrank}(\Pi \setminus \text{suspected})$ **do**

$\text{leader} := \text{maxrank}(\Pi \setminus \text{suspected});$

trigger $\langle \Omega, \text{Trust} \mid \text{leader} \rangle;$

Algorithm 2.9: Elect Lower Epoch

Implements: Omega for crash-recovery process abstraction.

EventualLeaderDetector, **instance** Ω . Rely on FLL instead of diamond P.

Uses:

FairLossPointToPointLinks, **instance** fll .

upon event $\langle \Omega, Init \rangle$ **do**

$epoch := 0$;

$store(epoch)$;

$candidates := \emptyset$;

trigger $\langle \Omega, Recovery \rangle$; // recovery

upon event $\langle \Omega, Recovery \rangle$ **do**

$leader := maxrank(\Pi)$;

trigger $\langle \Omega, Trust \mid leader \rangle$;

$delay := \Delta$;

$retrieve(epoch)$;

$epoch := epoch + 1$;

$store(epoch)$;

forall $p \in \Pi$ **do**

trigger $\langle fll, Send \mid p, [HEARTBEAT, epoch] \rangle$;

$candidates := \emptyset$;

$starttimer(delay)$;

upon event $\langle Timeout \rangle$ **do**

$newleader := select(candidates)$;

if $newleader \neq leader$ **then**

$delay := delay + \Delta$;

$leader := newleader$;

trigger $\langle \Omega, Trust \mid leader \rangle$;

forall $p \in \Pi$ **do**

trigger $\langle fll, Send \mid p, [HEARTBEAT, epoch] \rangle$;

$candidates := \emptyset$;

$starttimer(delay)$;

upon event $\langle fll, Deliver \mid q, [HEARTBEAT, ep] \rangle$ **do**

if exists $(s, e) \in candidates$ such that $s = q \wedge e < ep$ **then**

$candidates := candidates \setminus \{(q, e)\}$;

$candidates := candidates \cup (q, ep)$;

Byzantine Leader Detection (Partially Synchronous Model)

Detecting Arbitrary (Byzantine) Failures

- For Byzantine faulty processes, we **cannot rely on timeliness**:
 - A faulty process may respond **late**, **incorrectly**, or **selectively**
 - Missing or delayed responses do **not** reliably indicate failure
- Therefore:
 - **Timeout-based failure detection is insufficient**
 - Especially problematic for detecting a **Byzantine leader**

Byzantine Leader Detection

— Trust, but verify

Byzantine Leader Detection

— Idea: Give a leader a

chance to behave correctly

Byzantine Leader Detection — Trust, but Verify

- New leader is **temporarily trusted**
- Other processes **monitor leader behavior**
 - Correctness of actions
 - Progress within an expected time bound
- If *misbehavior* or *excessive slowness* is detected:
 - **Replace the leader**

Module 2.10: Interface and properties of the Byzantine eventual leader detector

Module:

Name: ByzantineLeaderDetector, **instance** *bld*.

Events:

Indication: $\langle bld, Trust \mid p \rangle$: Indicates that process p is trusted to be leader.

Request: $\langle bld, Complain \mid p \rangle$: Receives a complaint about process p .

Properties:

BLD1: *Eventual succession*: If more than f correct processes that trust some process p complain about p , then every correct process eventually trusts a different process than p .

BLD2: *Putsch resistance*: A correct process does not trust a new leader unless at least one correct process has complained against the previous leader.

BLD3: *Eventual agreement*: There is a time after which no two correct processes trust different processes.

Algorithm 2.10: Rotating Byzantine Leader Detection

Implements:

ByzantineLeaderDetector, **instance** *bld*.

Uses:

AuthPerfectPointToPointLinks, **instance** *al*.

upon event $\langle bld, Init \rangle$ **do**

$round := 1;$

$complainlist := [\perp]^N;$

$complained := \text{FALSE};$

trigger $\langle bld, Trust \mid leader(round) \rangle;$

upon event $\langle bld, Complain \mid p \rangle$ **such that** $p = leader(round)$ **and**

$complained = \text{FALSE}$ **do**

$complained := \text{TRUE};$

forall $q \in \Pi$ **do**

trigger $\langle al, Send \mid q, [\text{COMPLAINT}, round] \rangle;$

upon event $\langle al, Deliver \mid p, [\text{COMPLAINT}, r] \rangle$ **such that** $r = round$ **and**

$complainlist[p] = \perp$ **do**

$complainlist[p] := \text{COMPLAINT};$

if $\#(complainlist) > f \wedge complained = \text{FALSE}$ **then**

$complained := \text{TRUE};$

forall $q \in \Pi$ **do**

trigger $\langle al, Send \mid q, [\text{COMPLAINT}, round] \rangle;$

else if $\#(complainlist) > 2f$ **then**

$round := round + 1;$

$complainlist := [\perp]^N;$

$complained := \text{FALSE};$

trigger $\langle bld, Trust \mid leader(round) \rangle;$

Summary

- Asynchronous systems need **no time**
- Synchronous systems enable **perfect failure detection**
- Partial synchrony enables **eventual guarantees**
- Failure detectors are:
 - Fundamental
 - Inherently imperfect
 - Central to leader election

Joke time!

“This would be very fast
if the slowest node were
faster.”

“Timeouts? No need.
Bob will answer...
eventually.”

Distributed System Models

Combining Abstractions

- By combining:
 - a **process** abstraction
 - a **link** abstraction
 - a **failure detector** abstraction
- we obtain models for **synchronous** and **partially synchronous** systems

Combining Abstractions: Failure Models

- Fail-stop
- Fail-silent
- Fail-noisy
- Fail-recovery
- Fail-silent-arbitrary
- Fail-noisy-arbitrary

Combining Abstractions: Failure Models

	Algorithm Requirement	Process	Link	Timing	Quorum Req.
Fail-stop	Deterministic	Crash-stop	Perfect Links	$\mathcal{P}$	$N/2$ ($N \geq 2f+1$)
Fail-noisy	Deterministic	Crash-stop	Perfect Links	$\diamond \mathcal{P}$	$N/2$ ($N \geq 2f+1$)
Fail-silent	Deterministic	Crash-stop	Perfect Links	No FD	$N/2$ ($N \geq 2f+1$)
Fail-recovery	Deterministic	Crash-recovery	Stubborn Links (or Logged PL)	Ω	$N/2$ ($N \geq 2f+1$)
Fail-noisy-arbitrary	Correct Proc: Deterministic	Byzantine	Authenticated Perfect Links	$\diamond \Omega$	$N/3$ ($N \geq 3f+1$)
Fail-silent-arbitrary	Correct Proc: Deterministic	Byzantine	Authenticated Perfect Links	No FD	$N/3$ ($N \geq 3f+1$)
Randomized	Non-deterministic	Crash-stop			

Quorums

- A **quorum** is a minimum number of processes whose responses are sufficient to **safely perform an operation** in a distributed algorithm.
- Typically, quorums are chosen so that:
 - Any **two quorums intersect**
 - Information cannot be lost or contradicted across operations
- Common case: **majority quorums**

Quorums: Parameters

- Let:
 - **N** = number of processes in the system
 - **f** = maximum number of processes that may fail while the system still operates correctly
- Quorum sizes are chosen based on:
 - the **failure model**
 - the **process abstraction** (crash-stop vs. Byzantine)

Quorums: To tolerate Crash-stop Failures

- Majority quorum is sufficient
 - **$f + 1$ out of N**
- System size requirement:
 - **$N \geq 2f + 1$**
- Any two majority quorums ***intersect in at least one correct process***
- ➡ Consistency is preserved despite failures

Quorums with Byzantine Failures

- Faulty processes may **lie or equivocate**
- Two majority quorums may intersect **only in faulty processes**
 - ➡ Intersection alone is not sufficient

Quorums: To tolerate Byzantine Failures

- Stronger quorum sizes are required
 - **$2f + 1$ out of N**
- System size requirement:
 - **$N \geq 3f + 1$**
- Any two quorums **intersect in at least $f + 1$ correct processes**
-  Consistency is preserved despite arbitrary behavior

Measuring Performance of Algorithms

- Number of **messages** to complete an operation
- Number of **communication steps** (send/receive rounds)
- Number of **stable storage accesses**
 - (important in the crash-recovery model)

Designing Distributed Algorithms

- Optimize the **common case**
- **Degrade gracefully** as failures increase
- Maintain the same **asymptotic performance**
independent of the number of failures (up to the bound)

Questions?